```
{
  "name": "Tool-Process-CallRecordings-Relationships-Webhook",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "10577278-2424-43aa-aedd-abfb1c28faa8",
        "options": {}
      },
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 2.1,
      "position": [
        -16,
        208
      ],
      "id": "c26ab3b9-a0d9-4a46-9b2b-138f9aef0671",
      "name": "Webhook",
      "webhookId": "10577278-2424-43aa-aedd-abfb1c28faa8"
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "ab0b5622-838b-492f-bc5c-728d7d965b1f",
              "name": "caller_number_display",
              "value": "={{ $json.body.caller_number_format }}",
              "type": "string"
            },
            {
              "id": "d6bf53cd-f9e0-4674-b368-9d00712f2d11",
              "name": "caller_number_e164",
              "value": "={{ $json.body.caller_number_complete }}",
              "type": "string"
            },
            {
              "id": "858d3d43-de1f-4867-b570-b46763fe4530",
              "name": "callee_number_display",
              "value": "={{ $json.body.tracking_number_format }}",
              "type": "string"
            },
            {
              "id": "35c1db2f-a31e-48c2-a4dc-7a77f535c725",
              "name": "callee_number_e164",
              "value": "={{ $json.body.tracking_number }}",
              "type": "string"
            },
            {
              "id": "6c6e0740-6090-4da8-8a33-7179fffc3d28",
              "name": "call_id",
              "value": "={{ $json.body.id }}",
              "type": "number"
            },
            {
              "id": "2199f4ba-f2b4-401c-af21-cfbc3148e8c0",
              "name": "started_at",
              "value": "={{ new
Date($json.body.duration_period.gte).toISOString() }}",
              "type": "string"
```

```
            },
            {
              "id": "98ef1ed9-9116-4c68-8a5d-0b5acd668cad",
              "name": "ended_at",
              "value": "={{ new
Date($json.body.duration_period.lte).toISOString() }}",
              "type": "string"
            },
            {
              "id": "9de6d10c-0f79-4ddb-b5ce-012e6d57b127",
              "name": "duration",
              "value": "={{ $('Webhook').item.json.body.duration }}",
              "type": "number"
            },
            {
              "id": "1e4c93e2-4507-405b-b632-3ba834cf6e21",
              "name": "recording_url",
              "value": "={{ $json.body.audio }}",
              "type": "string"
            },
            {
              "id": "80087d6f-4f08-4831-93f7-ba82d4093147",
              "name": "direction",
              "value": "={{ $('Webhook').item.json.body.direction }}",
              "type": "string"
            },
            {
              "id": "15565fa2-b4af-4d63-8c90-6f2494a422b2",
              "name": "disposition",
              "value": "={{ $json.body.status }}",
              "type": "string"
            },
            {
              "id": "8e5d566f-7636-4917-9ae0-14c737fa52be",
              "name": "caller_email",
              "value": "={{ $json.body.emails }}",
              "type": "string"
            },
            {
              "id": "ba9e22d0-4f3d-4e1f-b751-cebfbe4b44fc",
              "name": "tracking_number",
              "value": "={{ $json.body.tracking_number }}",
              "type": "string"
            },
            {
              "id": "13761cea-0d23-42e7-b179-38850775dfbf",
              "name": "callee_name",
              "value": "={{ $json.body.tracking_label.replace(' Cell','') }}",
              "type": "string"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        592,
        192
```

```
      ],
      "id": "49c2c6c7-290f-4b80-a268-acc941a745b3",
      "name": "Get required fields"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "915789ff-d24a-4d58-bbc1-34f1187fbd35",
              "leftValue": "={{ $json.data.values[0][0].elementId }}",
              "rightValue": "",
              "operator": {
                "type": "string",
                "operation": "exists",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        1248,
        192
      ],
      "id": "2e24a9df-d615-4ca0-b771-ab27552fe042",
      "name": "Check if call node already exists1"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (c:Call {ctmCallId: $ctmCallId})\nRETURN
c\n`;\n\nconst parameters = {\n  ctmCallId: $input.first().json.call_id\n};\n\
nreturn [\n  {\n    json: {\n      statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        816,
        192
      ],
      "id": "3d39a637-61bf-4970-ac29-876e9e982efe",
      "name": "Prepare payload"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
```

```
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\": 
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1024,
        192
      ],
      "id": "0ccdebc7-bd2b-43a0-9f07-9924dece9428",
      "name": "Get Call node details",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMERGE (call:Call {ctmCallId: $ctmCallId})\nON 
CREATE SET\n  call.startedAt = datetime($startedAt),\n  call.endedAt = 
datetime($endedAt),\n  call.durationSecs = $durationSecs,\n  call.direction = 
$direction,\n  call.disposition = $disposition,\n  call.recordingUrl = 
$recordingUrl,\n  call.createdAt = datetime()\nRETURN call\n\n`;\n\nconst 
parameters = {\n  ctmCallId: $('Get required fields').first().json.call_id,\n 
startedAt:$('Get required fields').first().json.started_at,\n  endedAt:$('Get 
required fields').first().json.ended_at,\n  durationSecs:$('Get required 
fields').first().json.duration,\n  direction:$('Get required 
fields').first().json.direction,\n  disposition: $('Get required 
fields').first().json.disposition,\n  recordingUrl:$('Get required 
fields').first().json.recording_url,\n};\n\nreturn [\n  {\n    json: {\n 
statement: JSON.stringify(query),\n      parameters: JSON.stringify(parameters)\n 
}\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1360,
        320
      ],
      "id": "be9a8f42-c4b8-4696-9f1c-826c1a829b04",
      "name": "Prepare payload1"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\": 
{{ $json.parameters }}\n}\n",
```

```
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1568,
        320
      ],
      "id": "e7b55148-aa2a-4205-a4d8-51cb48e3bd9a",
      "name": "Create call node",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMERGE (phone:PhoneNumber {e164: $callerE164})\
nON CREATE SET\n  phone.displayNumber = $callerDisplay,\n  phone.source = 'CTM',\n
phone.firstSeen = datetime()\nON MATCH SET\n  phone.lastSeen = datetime()\nRETURN
phone.e164\n\n`;\n\nconst parameters = {\n  callerE164:$('Get required
fields').first().json.caller_number_e164,\n  callerDisplay:$('Get required
fields').first().json.callee_number_display\n};\n\nreturn [\n  {\n    json: {\n
statement: JSON.stringify(query),\n      parameters: JSON.stringify(parameters)\n
}\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1712,
        176
      ],
      "id": "b9f04ce6-a2ee-4f0a-aaa3-38585567164c",
      "name": "Prepare payload2"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1904,
        176
      ],
      "id": "a456fc65-fe88-4620-8d3e-7b1b1e289767",
      "name": "Caller PhoneNumber",
      "credentials": {
```

```
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMERGE (phone:PhoneNumber {e164: $callerE164})\
nON CREATE SET\n  phone.displayNumber = $callerDisplay,\n  phone.source = 'CTM',\n
phone.firstSeen = datetime()\nON MATCH SET\n  phone.lastSeen = datetime()\nRETURN
phone.e164\n\n`;\n\nconst parameters = {\n  callerE164:$('Get required
fields').first().json.callee_number_e164,\n  callerDisplay:$('Get required
fields').first().json.callee_number_display\n};\n\nreturn [\n  {\n    json: {\n
statement: JSON.stringify(query),\n      parameters: JSON.stringify(parameters)\n
}\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        2160,
        160
      ],
      "id": "03e25a14-5af1-488d-94c3-3a9c49b54ff6",
      "name": "Prepare payload3"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        2336,
        160
      ],
      "id": "3f8aad76-9871-4219-8ae1-48fef325685e",
      "name": "Callee PhoneNumber",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\
nMATCH (call:Call {ctmCallId: $ctmCallId})\nMERGE (phone)-[:ORIGINATED]->(call)\
```

n`;\n\nconst parameters = {\n callerE164:$('Get required fields').first().json.caller_number_e164,\n ctmCallId:$('Get required fields').first().json.call_id\n   \n};\n\nreturn [\n {\n   json: {\n statement: JSON.stringify(query),\n   parameters: JSON.stringify(parameters)\n }\n }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        2592,
        144
      ],
      "id": "6cd31ae2-195a-4f30-a394-24396dc1e548",
      "name": "Prepare payload4"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (call:Call {ctmCallId: $ctmCallId})\nMATCH (phone:PhoneNumber {e164: $calleeE164})\nMERGE (call)-[:REACHED {role: 'callee'}]->(phone)\n`;\n\nconst parameters = {\n ctmCallId:$('Get required fields').first().json.call_id,\n calleeE164:$('Get required fields').first().json.callee_number_e164\n};\n\nreturn [\n {\n   json: {\n statement: JSON.stringify(query),\n   parameters: JSON.stringify(parameters)\n }\n }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        3072,
        128
      ],
      "id": "9cf83b74-6a78-4ead-8438-b0ea0504247e",
      "name": "Prepare payload5"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n \"statement\": {{ $json.statement }},\n \"parameters\": {{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        2800,
        144
      ],
      "id": "556019af-d582-4e45-8f9a-53e673d7f8aa",
      "name": "Create ORIGINATED Relationship",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }

```
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        3296,
        128
      ],
      "id": "bdf9292a-6619-4581-9d33-ec9ed6e78179",
      "name": "Create REACHED Relationship",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "content": "## Receive event of call",
        "height": 384,
        "width": 272
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -48,
        64
      ],
      "typeVersion": 1,
      "id": "a36cef1a-d0ad-4f25-90ea-cf329fc481d2",
      "name": "Sticky Note"
    },
    {
      "parameters": {
        "content": "## get existing call node ",
        "height": 384,
        "width": 624,
        "color": 3
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        560,
        80
      ],
      "typeVersion": 1,
```

```
    "id": "cdcfb5f1-4cc1-46d2-b792-13e334b02346",
    "name": "Sticky Note2"
  },
  {
    "parameters": {
      "content": "## If call node not created yet create one",
      "height": 384,
      "width": 528,
      "color": 4
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      1184,
      80
    ],
    "typeVersion": 1,
    "id": "24b3d5dd-fb12-4767-9218-ffe6191e5163",
    "name": "Sticky Note3"
  },
  {
    "parameters": {
      "content": "## Create  caller phone number node",
      "height": 384,
      "width": 416,
      "color": 5
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      1712,
      80
    ],
    "typeVersion": 1,
    "id": "c484cc75-038d-4617-9bc9-b74e16afe4ab",
    "name": "Sticky Note4"
  },
  {
    "parameters": {
      "content": "## Create  callee phone number node",
      "height": 384,
      "width": 416,
      "color": 6
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      2112,
      80
    ],
    "typeVersion": 1,
    "id": "1e61f45f-2ef0-4131-8046-637308278c55",
    "name": "Sticky Note5"
  },
  {
    "parameters": {
      "content": "## Create ORIGINATED relationship",
      "height": 384,
      "width": 416,
      "color": 7
    },
    "type": "n8n-nodes-base.stickyNote",
```

```
      "position": [
        2528,
        80
      ],
      "typeVersion": 1,
      "id": "15a92792-174a-4ae5-b024-5346bdf042f8",
      "name": "Sticky Note6"
    },
    {
      "parameters": {
        "content": "## Create REACHED relationship",
        "height": 384,
        "width": 528
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        2944,
        80
      ],
      "typeVersion": 1,
      "id": "a5f7f21e-3cab-4c7c-a516-7e35040ae691",
      "name": "Sticky Note7"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\n
-[b:BELONGS_TO {current: true}]->(p:Person)\nRETURN p\n`;\n\nconst parameters = {\n
callerE164:$('Get required fields').first().json.caller_number_e164\n};\n\nreturn
[\n  {\n    json: {\n      statement: JSON.stringify(query),\n      parameters:
JSON.stringify(parameters)\n    }\n  }\n];\n\n\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        224,
        576
      ],
      "id": "57a35c31-4c52-497a-a6f8-83ab5c3ac009",
      "name": "Prepare payload6"
    },
    {
      "parameters": {
        "content": "## Identity Resolution - Caller ",
        "height": 432,
        "width": 2448,
        "color": 2
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        80,
        464
      ],
      "typeVersion": 1,
      "id": "40198ceb-2f42-446b-b754-2a466fe4e317",
      "name": "Sticky Note8"
    },
    {
      "parameters": {
        "method": "POST",
```

```
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\": 
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        464,
        576
      ],
      "id": "d45ac124-adf1-4808-80dd-8756dd674ff0",
      "name": "Check for a Direct Phone-to-Person Match",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "c624a958-75c8-4166-880f-d8430eccf0ce",
              "leftValue": "={{ $json.data.values[0][0].properties.email }}",
              "rightValue": true,
              "operator": {
                "type": "string",
                "operation": "exists",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        880,
        576
      ],
      "id": "c534bd54-face-45be-af21-a56004fc6d7c",
      "name": "Caller Direct Match?"
    },
```

```
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "14b65f0a-fb33-4348-93b6-71288ee13e73",
              "leftValue": "={{ $('Webhook').item.json.body.emails }}",
              "rightValue": "none",
              "operator": {
                "type": "array",
                "operation": "notEmpty",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        1008,
        752
      ],
      "id": "981bc39d-efb8-4c93-a68e-9086e805e01f",
      "name": "Has CTM Email?"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (p:Person {email: $email})\nRETURN
p.email AS email,\n       p.name AS name,\n       p.internal AS isInternal,\n
0.9 AS confidence,\n       'ctm_email' AS resolutionMethod,\n       true AS
resolved\n`;\n\nconst parameters = {\n  email: $('Get required
fields').first().json.caller_email\n  \n};\n\nreturn [\n  {\n    json: {\n
statement: JSON.stringify(query),\n      parameters: JSON.stringify(parameters)\n
}\n  }\n];"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1216,
        672
      ],
      "id": "bd5f2831-0611-420c-bc52-61bd8a305687",
      "name": "Prepare payload7"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
```

```
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "5d9bf39d-345f-4c96-8335-6844a244e252",
              "leftValue": "={{ $json.data.values[0][5] }}",
              "rightValue": "",
              "operator": {
                "type": "boolean",
                "operation": "true",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        1632,
        656
      ],
      "id": "c5b7970a-29c1-4c19-b7c7-0ed52f1e24f2",
      "name": "If"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMERGE (p:Person {email: $callerEmail})\nON
CREATE SET\n  p.name = $callerName,\n  p.createdAt = datetime()\n\nWITH p\n\nMATCH
(call:Call {ctmCallId: $ctmCallId})\n\nMERGE (p)-[i:INVOLVES]->(call)\nSET\n
i.role = 'caller',\n  i.resolvedBy = $method,\n  i.resolvedAt = datetime()\n\
nRETURN\n  p.email AS email,\n  p.name AS name,\n  p.isInternal AS isInternal\n`;\
n\nconst parameters = {\n  callerName:$('Webhook').first().json.body.cnam,\n
callerEmail: $input.first().json.fields.core.email.value,\n  ctmCallId: $('Get
required fields').first().json.call_id,\n  method: \"crm_email\"\n};\n\nreturn [\n
{\n    json: {\n      statement: JSON.stringify(query),\n      parameters:
JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        2832,
        576
      ],
      "id": "4a9eb7c3-47f8-402f-8f39-5b9d350c5a18",
      "name": "Prepare payload8"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $calleeE164})\n
-[b:BELONGS_TO {current: true}]->(p:Person)\nRETURN p.email as email , p.name as
name, p.internal as isInternal, b.confidence as cofidence, 'direct' as
resolutionMethod,true as resolved\n`;\n\nconst parameters = {\n  calleeE164:$('Get
required fields').first().json.callee_number_e164\n};\n\nreturn [\n  {\n    json:
{\n      statement: JSON.stringify(query),\n      parameters:
JSON.stringify(parameters)\n    }\n  }\n];\n\n\n"
```

```
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        192,
        1696
      ],
      "id": "0c9757b6-06fb-41d8-ba39-e2365c987a9d",
      "name": "Prepare payload9"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        480,
        1696
      ],
      "id": "7c53fa7d-75c0-4321-9cd7-1e1e8cd02616",
      "name": "Check for a Direct Phone-to-Person Match1",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "c624a958-75c8-4166-880f-d8430eccf0ce",
              "leftValue": "={{ $json.data.values[0][0].properties.email }}",
              "rightValue": true,
              "operator": {
                "type": "string",
                "operation": "exists",
                "singleValue": true
              }
            }
          ],
```

```
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        896,
        1696
      ],
      "id": "99d0acda-672a-4268-aa9b-936ec0f6b50e",
      "name": "Caller Direct Match?1"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "14b65f0a-fb33-4348-93b6-71288ee13e73",
              "leftValue": "={{ $('Webhook').item.json.body.emails }}",
              "rightValue": "none",
              "operator": {
                "type": "array",
                "operation": "notEmpty",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        1008,
        1872
      ],
      "id": "e894d4ce-7005-47a7-af6a-92bdff403d6d",
      "name": "Has CTM Email?1"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (p:Person {email: $email})\nRETURN
p.email AS email,\n       p.name AS name,\n       p.internal AS isInternal,\n
0.9 AS confidence,\n       'ctm_email' AS resolutionMethod,\n       true AS
resolved\n`;\n\nconst parameters = {\n  email: $node['Get required
fields'].json.body.caller_email\n  \n};\n\nreturn [\n  {\n    json: {\n
statement: JSON.stringify(query),\n      parameters: JSON.stringify(parameters)\n
}\n  }\n];"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
```

```
      "position": [
        1248,
        1856
      ],
      "id": "3c5aacff-7579-4add-a24e-e1f9b7ae0986",
      "name": "Prepare payload10"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1440,
        1856
      ],
      "id": "3afb450a-75ea-40af-a1d9-94f422c22e2c",
      "name": "Caller: CTM Email Match2",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "5d9bf39d-345f-4c96-8335-6844a244e252",
              "leftValue": "={{ $json.data.values[0][5] }}",
              "rightValue": "",
              "operator": {
                "type": "boolean",
                "operation": "true",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
```

```
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        1712,
        1840
      ],
      "id": "4e1dac21-dd47-483e-8ab6-7fb15ad911cf",
      "name": "If1"
    },
    {
      "parameters": {
        "content": "## Identity Resolution - Callee",
        "height": 432,
        "width": 2448,
        "color": 4
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        112,
        1632
      ],
      "typeVersion": 1,
      "id": "eba2ec36-b396-4bc7-af09-678d3d6d8156",
      "name": "Sticky Note9"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\": 
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        3072,
        576
      ],
      "id": "4433559d-16f6-4f4a-9fc2-52491b1ca884",
      "name": "Link Caller to Call",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "content": "## Linking caller to a person",
        "height": 432,
        "width": 944,
```

```
        "color": 3
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        2560,
        448
      ],
      "typeVersion": 1,
      "id": "542ab5c6-d48d-4d03-99d7-2fdd9f0db5d1",
      "name": "Sticky Note10"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMERGE (p:Person {email: $calleeEmail})\nON
CREATE SET\n  p.name = $calleeName,\n  p.createdAt = datetime()\n\nWITH p\n\nMATCH
(call:Call {ctmCallId: $ctmCallId})\n\nMERGE (p)-[i:INVOLVES]->(call)\nSET\n
i.role = 'callee',\n  i.resolvedBy = $method,\n  i.resolvedAt = datetime()\n\
nRETURN\n  p.email AS email,\n  p.name AS name,\n  p.isInternal AS isInternal\n`;\
n\nconst parameters = {\n  calleeName:$('Get required
fields').first().json.callee_name,\n  calleeEmail: $('get
output1').first().json.output.contact_info.work_email,\n  ctmCallId: $('Get
required fields').first().json.call_id,\n  method: \"crm_email\"\n};\n\nreturn [\n
{\n    json: {\n      statement: JSON.stringify(query),\n      parameters:
JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        2976,
        1760
      ],
      "id": "ad37454e-e9d4-409e-950a-47dbedb2aaf1",
      "name": "Prepare payload12"
    },
    {
      "parameters": {
        "content": "## Linking callee to a person",
        "height": 432,
        "width": 944,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        2560,
        1632
      ],
      "typeVersion": 1,
      "id": "186fb25c-e2a8-4fc9-ae8e-8e6327cf382e",
      "name": "Sticky Note11"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
```

{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        3280,
        1760
      ],
      "id": "c34b5956-deba-4b38-91c4-fc9a6d395817",
      "name": "Link Callee to Call",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "content": "## Qualification for Metrics:",
        "height": 480,
        "width": 592,
        "color": 6
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        128,
        2848
      ],
      "typeVersion": 1,
      "id": "62fe1b5c-a624-477b-9ba5-e1ac08adaac2",
      "name": "Sticky Note12"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "7283e66d-7b35-4f43-9e3b-22edb22de4f1",
              "leftValue": "={{ $('Get required
fields').first().json.disposition }}",
              "rightValue": "answered",
              "operator": {
                "type": "string",
                "operation": "equals"
              }
            },
            {
              "id": "f248e489-e5ec-4f0b-ad77-0246918aed31",
              "leftValue": "={{ $('Webhook').first().json.body.duration }}",
              "rightValue": 30,

```
            "operator": {
              "type": "number",
              "operation": "gte"
            }
          }
        ],
        "combinator": "and"
      },
      "options": {}
    },
    "type": "n8n-nodes-base.if",
    "typeVersion": 2.3,
    "position": [
      256,
      3040
    ],
    "id": "99828f3b-59f4-43c6-86c6-12426478402f",
    "name": "Qualify for metrics?"
  },
  {
    "parameters": {
      "content": "## Qualification for RAG / Transcript Processing:",
      "height": 560,
      "width": 336
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      128,
      3328
    ],
    "typeVersion": 1,
    "id": "929e0c48-003b-4a8a-9f3d-263f64dc3517",
    "name": "Sticky Note13"
  },
  {
    "parameters": {
      "numberInputs": 5
    },
    "type": "n8n-nodes-base.merge",
    "typeVersion": 3.2,
    "position": [
      2624,
      528
    ],
    "id": "129bb6ca-6432-4a55-b454-3ae18d84af8e",
    "name": "Merge"
  },
  {
    "parameters": {
      "numberInputs": 5
    },
    "type": "n8n-nodes-base.merge",
    "typeVersion": 3.2,
    "position": [
      2624,
      1712
    ],
    "id": "718d8b3a-e957-4581-a9b8-59b89e09e650",
    "name": "Merge1"
```

```
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "7283e66d-7b35-4f43-9e3b-22edb22de4f1",
              "leftValue": "={{ $('Get required
fields').first().json.disposition }}",
              "rightValue": "answered",
              "operator": {
                "type": "string",
                "operation": "equals"
              }
            },
            {
              "id": "f248e489-e5ec-4f0b-ad77-0246918aed31",
              "leftValue": "={{ $('Webhook').item.json.body.duration }}",
              "rightValue": 60,
              "operator": {
                "type": "number",
                "operation": "gte"
              }
            },
            {
              "id": "f23b5f58-dfa9-4fd9-9f4e-8a6bd159930f",
              "leftValue": "={{ $('Webhook').item.json.body.transcription_text }}",
              "rightValue": "false",
              "operator": {
                "type": "string",
                "operation": "exists",
                "singleValue": true
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        224,
        3424
      ],
      "id": "36afa97c-b34e-4105-945d-2afa8b81b9c6",
      "name": "Qualify for RAG?"
    },
    {
      "parameters": {
        "mode": "raw",
        "jsonOutput": "{\n  \"message\": \"no further action\"\n\n}\n",
        "options": {}
```

```
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        368,
        3200
      ],
      "id": "c356b60c-332b-4d61-ac26-22366ceace2b",
      "name": "status"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
            {
              "id": "a12db16a-6a60-455b-8739-a674bd6bc5d0",
              "leftValue": "={{ $
('Webhook').first().json.body.transcription_text }}",
              "rightValue": "",
              "operator": {
                "type": "string",
                "operation": "contains"
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        960,
        3408
      ],
      "id": "26e7801a-14d8-4ad4-b826-a054d9c69ec8",
      "name": "if transcript found?"
    },
    {
      "parameters": {
        "jsCode": "// Input transcript\nconst transcript = $
('Webhook').first().json.body.transcription_text || \"\";\n\n// Regex patterns for
sensitive data\nconst patterns = {\n  SSN: /\\b\\d{3}-\\d{2}-\\d{4}\\b/g,\n
CREDIT_CARD: /\\b(?:\\d[ -]*?){13,16}\\b/g,\n  PHONE: /\\b(?:\\+?\\d{1,3}[-.\\s]?)?
(?:\\(?\\d{3}\\)?[-.\\s]?)\\d{3}[-.\\s]?\\d{4}\\b/g,\n  EMAIL: /\\b[A-Z0-9._%+-]
+@[A-Z0-9.-]+\\.[A-Z]{2,}\\b/gi,\n};\n\n// Audit log\nconst auditLog = [];\n\n//
Redaction function\nlet redactedTranscript = transcript;\n\nfor (const [type,
regex] of Object.entries(patterns)) {\n  const matches = transcript.match(regex);\
n\n  if (matches) {\n    matches.forEach(match => {\n      auditLog.push({\n
type,\n        value: match.replace(/.(?=.{4})/g, \"*\"), // partially mask for
audit\n        timestamp: new Date().toISOString(),\n      });\n    });\n\n
redactedTranscript = redactedTranscript.replace(\n      regex,\n      `[REDACTED_$
{type}]`\n    );\n  }\n}\n\n// Return result\nreturn [\n  {\n    json: {\n
```

redactedTranscript,\n   redactionAudit: auditLog,\n   redactionCount: auditLog.length,\n   }\n }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1296,
        3392
      ],
      "id": "6c364afb-a157-4a51-bba1-457955af690a",
      "name": "Redact Sensitive Information Using Pattern Matching"
    },
    {
      "parameters": {
        "modelId": {
          "__rl": true,
          "value": "gpt-5.2-chat-latest",
          "mode": "list",
          "cachedResultName": "GPT-5.2-CHAT-LATEST"
        },
        "responses": {
          "values": [
            {
              "content": "={{ $json.redactedTranscript }}"
            },
            {
              "role": "system",
              "content": "=#Redact Additional PII Using AI-Based Recognition\n\nYou are an AI system specialized in **privacy protection and data sanitization**. \nYour task is to **automatically detect and redact all additional Personally Identifiable Information (PII)** from the provided text using AI-based recognition techniques.\n\n## Instructions\n- Analyze the entire input carefully, including structured and unstructured text.\n- Identify **explicit and implicit PII**, even if not labeled.\n- Redact detected PII consistently using standardized placeholders.\n- Preserve the original structure, formatting, and readability of the text.\n- Do **not** infer or generate new personal data.\n- Do **not** modify non-PII content.\n\n## PII to Detect and Redact (Non-Exhaustive)\n- **Personal Names** (full, partial, nicknames)\n- **Phone Numbers** (mobile, landline, VoIP)\n- **Email Addresses**\n- **Physical Addresses** (street, city, state, postal code, country)\n- **Government-Issued IDs** (CNIC, SSN, passport, driver’s license)\n- **Financial Information** (bank accounts, credit/debit cards, IBAN)\n- **Online Identifiers** (usernames, IP addresses, device IDs)\n- **Biometric or Health Identifiers**\n- **Organization-Specific IDs** (employee ID, customer ID)\n- **Voice, Call, or Recording References** tied to an identifiable individual\n- **Any other data that can reasonably identify a person**\n\n## Redaction Rules\n- Replace each PII type with a clear placeholder:\n - `[REDACTED_NAME]`\n - `[REDACTED_PHONE]`\n - `[REDACTED_EMAIL]`\n - `[REDACTED_ADDRESS]`\n - `[REDACTED_ID]`\n - `[REDACTED_FINANCIAL]`\n - `[REDACTED_IP]`\n- Use the **same placeholder for the same PII type throughout**.\n- If unsure whether data is PII, **err on the side of redaction**.\n\n## Output Requirements\n- Return **only the redacted version** of the input text.\n- Do not include explanations, metadata, or summaries.\n- Maintain original line breaks, punctuation, and formatting.\n\n"
            }
          ]
        },
        "builtInTools": {},
        "options": {}
      },
      "type": "@n8n/n8n-nodes-langchain.openAi",

```
      "typeVersion": 2.1,
      "position": [
        1504,
        3392
      ],
      "id": "f281978b-b788-4b0f-89b1-7d5c6c490843",
      "name": "Redact Additional PII Using AI-Based Recognition",
      "credentials": {
        "openAiApi": {
          "id": "fsHBo55IPrWpeJ8r",
          "name": "OpenAi account"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "promptType": "define",
        "text": "={{ $json.output[0].content[0].text }}",
        "options": {
          "systemMessage": "=You are a conversation summarizer. Extract all
information from the transcript **strictly in JSON format** using the following
structure:\n\n{\n  \"summary\": \"2-3 sentence overview\",\n  \"keyPoints\":
[\"Main topic 1\", \"Main topic 2\"],\n  \"actionItems\": [\"Follow-up action
1\", \"Follow-up action 2\"],\n  \"decisions\": [\"Decision 1\", \"Decision 2\"],\n
\"openQuestions\": [\"Question 1\", \"Question 2\"],\n  \"sentiment\": \"positive|
neutral|negative|mixed\",\n  \"salesIntelligence\": {\n    \"painPoints\": [\n
{\n        \"statement\": \"\",\n        \"category\": \"operational|financial|
technical|strategic\",\n        \"severity\": \"high|medium|low\",\
n        \"speakerRole\": \"\"\n      }\n    ],\n    \"budgetSignals\": [\n      {\
n        \"statement\": \"\",\n        \"type\": \"explicit_amount|range|
constraint|approval_needed\",\n        \"value\": \"\",\
n        \"confidence\": \"\"\n      }\n    ],\n    \"timelineIndicators\": [\n
{\n        \"statement\": \"\",\n        \"type\": \"deadline|urgency|delay|
dependency\",\n        \"date_reference\": \"\"\n      }\n    ],\
n    \"stakeholders\": [\n      {\n        \"name\": \"\",\
n        \"role\": \"\",\n        \"influence\": \"decision_maker|influencer|user|
blocker\",\n        \"sentiment\": \"\"\n      }\n    ],\n    \"objections\": [\n
{\n        \"statement\": \"\",\n        \"category\": \"price|timing|competition|
internal|technical\",\n        \"addressed\": true\n      }\n    ],\
n    \"buyingSignals\": [\n      {\n        \"statement\": \"\",\
n        \"strength\": \"strong|moderate|weak\"\n      }\n    ],\
n    \"nextSteps\": [\n      {\n        \"action\": \"\",\n        \"owner\": \"us|
them|mutual\",\n        \"due\": \"\"\n      }\n    ],\n    \"competitorMentions\":
[\"competitor1\", \"competitor2\"]\n  }\n}\n\n**Do not add any explanation or text
outside this JSON.**\n"
        }
      },
      "type": "@n8n/n8n-nodes-langchain.agent",
      "typeVersion": 3.1,
      "position": [
        1872,
        3376
      ],
      "id": "e03e924f-8964-4c29-bba5-9aab75f6e35a",
      "name": "AI Summary & Sales Intelligence Extraction",
      "onError": "continueErrorOutput"
    },
    {
```

```
    "parameters": {
      "model": {
        "__rl": true,
        "value": "gpt-5.2-chat-latest",
        "mode": "list",
        "cachedResultName": "gpt-5.2-chat-latest"
      },
      "builtInTools": {},
      "options": {}
    },
    "type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
    "typeVersion": 1.3,
    "position": [
      1856,
      3632
    ],
    "id": "ae998db1-0f25-4908-a828-c2f5e8a38b26",
    "name": "OpenAI Chat Model",
    "credentials": {
      "openAiApi": {
        "id": "fsHBo55IPrWpeJ8r",
        "name": "OpenAi account"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "content": "## Qualification for RAG / Transcript Processing:",
      "height": 560,
      "width": 2032,
      "color": 4
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      464,
      3328
    ],
    "typeVersion": 1,
    "id": "5c5eae07-502e-42a6-be63-30fddeece110",
    "name": "Sticky Note14"
  },
  {
    "parameters": {
      "textSplittingMode": "custom",
      "options": {}
    },
    "type": "@n8n/n8n-nodes-langchain.documentDefaultDataLoader",
    "typeVersion": 1.1,
    "position": [
      2880,
      3552
    ],
    "id": "f4b36059-0266-4333-b627-5765775be045",
    "name": "Default Data Loader",
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
```

"chunkSize": 600,
"chunkOverlap": 100,
"options": {}
},
"type": "@n8n/n8n-nodes-langchain.textSplitterRecursiveCharacterTextSplitter",
"typeVersion": 1,
"position": [
2880,
3728
],
"id": "6d931fb8-3366-4441-b046-a766a20995b5",
"name": "Recursive Character Text Splitter",
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\nOPTIONAL MATCH (phone)-[existing:BELONGS_TO {current: true}]->(existingPerson:Person)\n\nWITH phone, existing, existingPerson,\n CASE\n   WHEN existing IS NULL THEN true\n   WHEN $newConfidence > existing.confidence THEN true\n   ELSE false\n END AS shouldAssign\n\n// Close the old current relationship if we should assign a new one\nFOREACH (_ IN CASE WHEN shouldAssign AND existing IS NOT NULL THEN [1] ELSE [] END |\n SET existing.current = false,\n existing.validTo = datetime()\n)\n\n// Create a new relationship if needed\nWITH phone, shouldAssign\nMATCH (person:Person {email: $personEmail})\nWHERE shouldAssign\nMERGE (phone)-[b:BELONGS_TO {current:true}]->(person)\nON CREATE SET\n b.current = true,\n b.source = $source,\n b.confidence = $newConfidence,\n b.validFrom = datetime(),\n b.verified = false\n\nRETURN shouldAssign, person.email\n`;\n\nconst parameters = {\n callerE164: $('Get required fields').first().json.caller_number_e164,\n personEmail: $('Webhook').first().json.body.caller_email,\n source: 'CTM',\n newConfidence: 1.0\n};\n\nreturn [\n {\n   json: {\n   statement: JSON.stringify(query),\n parameters: JSON.stringify(parameters)\n   }\n }\n];\n"
},
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [
1056,
480
],
"id": "b99651f1-8f01-43d9-b6d9-c19103720ca5",
"name": "Prepare payload13"
},
{
"parameters": {
"method": "POST",
"url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
"authentication": "genericCredentialType",
"genericAuthType": "httpBasicAuth",
"sendBody": true,
"specifyBody": "json",
"jsonBody": "={\n \"statement\": {{ $json.statement }},\n \"parameters\": {{ $json.parameters }}\n}\n",
"options": {}
},
"type": "n8n-nodes-base.httpRequest",
"typeVersion": 4.3,
"position": [

```
        1456,
        480
      ],
      "id": "740437e1-6b4e-470c-8644-a1e6f9aaf126",
      "name": "Create Phone Ownership of Caller",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\
nOPTIONAL MATCH (phone)-[existing:BELONGS_TO {current: true}]-
>(existingPerson:Person)\n\nWITH phone, existing, existingPerson,\n  CASE\n    WHEN
existing IS NULL THEN true\n    WHEN $newConfidence > existing.confidence THEN
true\n    ELSE false\n  END AS shouldAssign\n\nFOREACH (_ IN CASE WHEN shouldAssign
AND existing IS NOT NULL THEN [1] ELSE [] END |\n  SET existing.current = false,\n
existing.validTo = datetime()\n)\n\nWITH phone, shouldAssign\nMATCH (person:Person
{email: $personEmail})\nWHERE shouldAssign\nMERGE (phone)-[b:BELONGS_TO]->(person)\
nON CREATE SET\n  b.current = true,\n  b.source = $source,\n  b.confidence =
$newConfidence,\n  b.validFrom = datetime(),\n  b.verified = false\n\nRETURN
shouldAssign, person.email\n`;\n\nconst parameters = {\n  callerE164:$node['Get
required fields'].json.caller_number_e164,\n  personEmail:$node['Get required
fields'].json.body.caller_email,\n  source:'CTM',\n  newConfidence:0.9\n  \n};\n\
nreturn [\n  {\n    json: {\n      statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1808,
        592
      ],
      "id": "4d65afa5-c109-4403-922f-5ce5f4959ee6",
      "name": "Prepare payload14"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        2128,
        544
      ],
      "id": "fd72ef17-0c9f-47e3-90ae-4b53d6452bbb",
```

```
      "name": "Create Phone Ownership of Caller1",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\
nOPTIONAL MATCH (phone)-[existing:BELONGS_TO {current: true}]-
>(existingPerson:Person)\n\nWITH phone, existing, existingPerson,\n  CASE\n    WHEN
existing IS NULL THEN true\n    WHEN $newConfidence > existing.confidence THEN
true\n    ELSE false\n  END AS shouldAssign\n\nFOREACH (_ IN CASE WHEN shouldAssign
AND existing IS NOT NULL THEN [1] ELSE [] END |\n  SET existing.current = false,\n
existing.validTo = datetime()\n)\n\nWITH phone, shouldAssign\nMATCH (person:Person
{email: $personEmail})\nWHERE shouldAssign\nMERGE (phone)-[b:BELONGS_TO]->(person)\
nON CREATE SET\n  b.current = true,\n  b.source = $source,\n  b.confidence =
$newConfidence,\n  b.validFrom = datetime(),\n  b.verified = false\n\nRETURN
shouldAssign, person.email\n`;\n\nconst parameters = {\n  callerE164: $node['Get
required   fields'].json.caller_number_e164,\n  personEmail:
$input.first().json.emails,\n  source:'CRM',\n  newConfidence:0.95\n  \n};\n\
nreturn [\n  {\n    json: {\n      statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        880,
        1040
      ],
      "id": "e7a2396c-10bb-4b60-9e5a-f7c217cc25ce",
      "name": "Prepare payload15"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1152,
        1040
      ],
      "id": "770838a8-88d4-418a-a0ea-609acd86a0cc",
      "name": "Create Phone Ownership of Caller2",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
```

```
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\
nOPTIONAL MATCH (phone)-[existing:BELONGS_TO {current: true}]-
>(existingPerson:Person)\n\nWITH phone, existing, existingPerson,\n  CASE\n    WHEN
existing IS NULL THEN true\n    WHEN $newConfidence > existing.confidence THEN
true\n    ELSE false\n  END AS shouldAssign\n\n// Close the old current
relationship if we should assign a new one\nFOREACH (_ IN CASE WHEN shouldAssign
AND existing IS NOT NULL THEN [1] ELSE [] END |\n  SET existing.current = false,\n
existing.validTo = datetime()\n)\n\n// Create a new relationship if needed\nWITH
phone, shouldAssign\nMATCH (person:Person {email: $personEmail})\nWHERE
shouldAssign\nMERGE (phone)-[b:BELONGS_TO {current:true}]->(person)\nON CREATE SET\
n  b.current = true,\n  b.source = $source,\n  b.confidence = $newConfidence,\n
b.validFrom = datetime(),\n  b.verified = false\n\nRETURN shouldAssign,
person.email\n`;\n\nconst parameters = {\n  callerE164: $('Get required
fields').first().json.callee_number_e164,\n  personEmail: $node['Get required
fields'].json.body.caller_email,\n  source: 'CTM',\n  newConfidence: 1.0\n};\n\
nreturn [\n  {\n    json: {\n      statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1184,
        1648
      ],
      "id": "d9d2dd02-935f-491d-aec7-c7ea9b459f08",
      "name": "Prepare payload16"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1440,
        1648
      ],
      "id": "a53e24eb-3dd1-4ece-a57a-6253035ec417",
      "name": "Create Phone Ownership of Callee",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
```

```
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $calleeE164})\
nOPTIONAL MATCH (phone)-[existing:BELONGS_TO {current: true}]-
>(existingPerson:Person)\n\nWITH phone, existing, existingPerson,\n  CASE\n    WHEN
existing IS NULL THEN true\n    WHEN $newConfidence > existing.confidence THEN
true\n    ELSE false\n  END AS shouldAssign\n\nFOREACH (_ IN CASE WHEN shouldAssign
AND existing IS NOT NULL THEN [1] ELSE [] END |\n  SET existing.current = false,\n
existing.validTo = datetime()\n)\n\nWITH phone, shouldAssign\nMATCH (person:Person
{email: $personEmail})\nWHERE shouldAssign\nMERGE (phone)-[b:BELONGS_TO]->(person)\
nON CREATE SET\n  b.current = true,\n  b.source = $source,\n  b.confidence =
$newConfidence,\n  b.validFrom = datetime(),\n  b.verified = false\n\nRETURN
shouldAssign, person.email\n`;\n\nconst parameters = {\n  calleeE164:$node['Get
required fields'].json.callee_number_e164,\n  personEmail:$node['Get required
fields'].json.body.caller_email,\n  source:'CTM',\n  newConfidence:0.9\n  \n};\n\
nreturn [\n  {\n    json: {\n      statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1984,
        1760
      ],
      "id": "3bb9b2e6-a5f2-4900-80ee-6cc13fd27651",
      "name": "Prepare payload17"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        2176,
        1728
      ],
      "id": "a29ad090-2360-4230-9836-6cc83d0c0935",
      "name": "Create Phone Ownership of Caller3",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
```

```
      "parameters": {
        "content": "## Update RAG References",
        "height": 400,
        "width": 688,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        128,
        3936
      ],
      "typeVersion": 1,
      "id": "a2362e9a-104f-4c8f-b4e1-6a1b6582b284",
      "name": "Sticky Note15"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (call:Call {ctmCallId: $ctmCallId})\nSET
call.transcriptDocId = $transcriptDocId,\n    call.summaryDocId = $summaryDocId,\n
call.sentiment = $sentiment,\n    call.summary = $summaryText,\n    call.keyPoints
= $keyPoints,\n    call.actionItemCount = size($actionItems),\n
call.salesIntelligence = $salesIntelligenceJson,\n    call.hasSalesIntel = true,\n
call.intelligenceExtractedAt = datetime(),\n    call.ragProcessedAt = datetime()\
nRETURN call.ctmCallId\n`;\n\nconst parameters = {\n  ctmCallId:$
('Webhook').first().json.body.id,\n  transcriptDocId:
$input.first().json.transcriptDocId,\n  summaryDocId:
$input.first().json.summaryDocId,\n  sentiment: $input.first().json.sentiment,\n
summaryText: $('Parse Json').first().json.summary,\n  keyPoints: $('Parse
Json').first().json.keyPoints,\n  actionItems: $('Parse
Json').first().json.actionItems,\n  salesIntelligenceJson: JSON.stringify($('Parse
Json').first().json.salesIntelligence)\n};\n\nreturn [\n  {\n    json: {\n
statement: JSON.stringify(query),\n      parameters: JSON.stringify(parameters)\n
}\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        416,
        4048
      ],
      "id": "9c92eef2-01bf-4a5e-8c47-7bbe5a8e4066",
      "name": "Prepare payload19"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        640,
```

```
    4048
  ],
  "id": "859c40fc-a970-4dac-a9f3-abb1e783cb89",
  "name": "Update RAG References on Call node",
  "credentials": {
    "httpBasicAuth": {
      "id": "nWJCZ6f1qT459ztQ",
      "name": "neo4j"
    }
  },
  "onError": "continueErrorOutput"
},
{
  "parameters": {
    "assignments": {
      "assignments": [
        {
          "id": "59b564af-bdef-4681-bb61-b7b276c9de2d",
          "name": "transcript",
          "value": "={{ $('Webhook').first().json.body.transcription_text }}",
          "type": "string"
        }
      ]
    },
    "options": {}
  },
  "type": "n8n-nodes-base.set",
  "typeVersion": 3.4,
  "position": [
    2512,
    3424
  ],
  "id": "4cbc016f-efb1-4a34-8c34-8ac72d5608cd",
  "name": "Get transcript"
},
{
  "parameters": {
    "content": "## Insert all transcript into Vector DB",
    "height": 560,
    "width": 624
  },
  "type": "n8n-nodes-base.stickyNote",
  "position": [
    2496,
    3328
  ],
  "typeVersion": 1,
  "id": "570bd0b1-e2e5-4853-9685-761fb80397d1",
  "name": "Sticky Note16"
},
{
  "parameters": {
    "mode": "insert",
    "pineconeIndex": {
      "__rl": true,
      "value": "calltranscript",
      "mode": "list"
    },
    "options": {}
```

```
      },
      "type": "@n8n/n8n-nodes-langchain.vectorStorePinecone",
      "typeVersion": 1.3,
      "position": [
        2736,
        3424
      ],
      "id": "b68a646e-f8f1-405c-bad7-62329f8dd6e7",
      "name": "Insert transcript into calltranscript db",
      "credentials": {
        "pineconeApi": {
          "id": "MVjHI88pENPm3hEn",
          "name": "PineconeApi account (CTM Call Processing Project)"
        }
      }
    },
    {
      "parameters": {
        "options": {}
      },
      "type": "@n8n/n8n-nodes-langchain.embeddingsOpenAi",
      "typeVersion": 1.2,
      "position": [
        2688,
        3632
      ],
      "id": "9d73b4fb-fd4a-4e12-b9b1-eaf1c50d7c84",
      "name": "Embeddings OpenAI",
      "credentials": {
        "openAiApi": {
          "id": "fsHBo55IPrWpeJ8r",
          "name": "OpenAi account"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "return items.map(item => {\n  return {\n    json:
JSON.parse(item.json.output)\n  };\n});\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        2240,
        3360
      ],
      "id": "9520f502-3a3d-42e9-9b6f-45b2ee3692e0",
      "name": "Parse Json"
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "a6e609a1-211a-4382-948d-278ffbb1c188",
              "name": "summary",
              "value": "={{ $('Parse Json').first().json.summary }}",
              "type": "string"
```

```
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        3168,
        3424
      ],
      "id": "3d1a7b95-ae17-40ae-b6bd-87e009d332a2",
      "name": "Get summary",
      "alwaysOutputData": true,
      "executeOnce": true
    },
    {
      "parameters": {
        "textSplittingMode": "custom",
        "options": {}
      },
      "type": "@n8n/n8n-nodes-langchain.documentDefaultDataLoader",
      "typeVersion": 1.1,
      "position": [
        3520,
        3536
      ],
      "id": "2b7a8249-269d-4e63-aed0-66b621343ce0",
      "name": "Default Data Loader1",
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "chunkSize": 600,
        "chunkOverlap": 100,
        "options": {}
      },
      "type": "@n8n/n8n-nodes-
langchain.textSplitterRecursiveCharacterTextSplitter",
      "typeVersion": 1,
      "position": [
        3520,
        3712
      ],
      "id": "8018fdd4-5cf0-4f2e-9596-c2fc0ac25f36",
      "name": "Recursive Character Text Splitter1",
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "content": "## Insert summary of transcript  into Vector DB",
        "height": 560,
        "width": 672,
        "color": 3
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        3120,
        3328
```

```
    ],
    "typeVersion": 1,
    "id": "5a50145c-82e1-4c65-a1de-4fbf96d41e01",
    "name": "Sticky Note17"
  },
  {
    "parameters": {
      "options": {}
    },
    "type": "@n8n/n8n-nodes-langchain.embeddingsOpenAi",
    "typeVersion": 1.2,
    "position": [
      3280,
      3648
    ],
    "id": "ddfccce1-a11d-4600-8394-044aecce36fd",
    "name": "Embeddings OpenAI1",
    "credentials": {
      "openAiApi": {
        "id": "fsHBo55IPrWpeJ8r",
        "name": "OpenAi account"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "mode": "insert",
      "pineconeIndex": {
        "__rl": true,
        "value": "transcriptsummary",
        "mode": "list",
        "cachedResultName": "transcriptsummary"
      },
      "options": {}
    },
    "type": "@n8n/n8n-nodes-langchain.vectorStorePinecone",
    "typeVersion": 1.3,
    "position": [
      3376,
      3424
    ],
    "id": "e2b834b2-9d38-4e01-a6f6-8d06cc8df9f7",
    "name": "Insert summary into summary db",
    "credentials": {
      "pineconeApi": {
        "id": "MVjHI88pENPm3hEn",
        "name": "PineconeApi account (CTM Call Processing Project)"
      }
    }
  },
  {
    "parameters": {
      "assignments": {
        "assignments": [
          {
            "id": "919b806d-2366-4350-b1f4-6dee1837bbde",
            "name": "transcriptDocId",
            "value": "=transcript_{{ $('Webhook').first().json.body.id }}",
```

```
              "type": "string"
            },
            {
              "id": "a79d4a3a-a35b-4bb5-9741-8a98ba9acea6",
              "name": "summaryDocId",
              "value": "=summary_{{ $('Webhook').first().json.body.id }}",
              "type": "string"
            },
            {
              "id": "bfb83d66-57be-4c37-b460-19cc88b9c2b5",
              "name": "sentiment",
              "value": "={{ $('Parse Json').first().json.sentiment }}",
              "type": "string"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        208,
        4048
      ],
      "id": "71c2575a-477c-42f3-9115-81c30b7c389a",
      "name": "Set transcript and summary ids"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (internal:Person {email:
$internalEmail})\n-[r:COMMUNICATES_WITH]-\n(external:Person {email:
$externalEmail})\nRETURN r.count_30d AS count_30d,\n       r.decayed AS decayed,\n
r.last_seen AS last_seen\nLIMIT 1\n`;\n\nconst parameters = {\n  internalEmail:
$input.first().json.value2,\n  externalEmail:$input.first().json.value1 \n    \n  \
n};\n\nreturn [\n  {\n    json: {\n      statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n    }\n  }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1392,
        3008
      ],
      "id": "70265ec8-a0ae-4342-92d3-34a4bf0cf4b5",
      "name": "Prepare payload20"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (internal:Person {email:
$internalEmail})\nMATCH (external:Person {email: $externalEmail})\nMERGE
(internal)-[r:COMMUNICATES_WITH]-(external)\nON CREATE SET\n  r.totalCalls = 1,\n
r.count_30d = 1,\n  r.decayed = 1,\n  r.tieStrength = 1,\n  r.last_seen =
datetime()\nON MATCH SET\n  r.totalCalls = coalesce(r.totalCalls, 0) + 1,\n
r.count_30d = $newCount30d,\n  r.decayed = $newDecayed,\n  r.tieStrength =
$newTieStrength,\n  r.last_seen = datetime()\n\n`;\n\nconst parameters = {\n
internalEmail:$('Set values').first().json.value2,\n  externalEmail:$('Set
values').first().json.value2,\n  newCount30d:$input.first().json.newDecayed,\n
newDecayed:$input.first().json.newCount30d,\n  newTieStrength:
$input.first().json.newTieStrength\n  \n    \n  \n};\n\nreturn [\n  {\n    json: {\
```

n   statement: JSON.stringify(query),\n   parameters:
JSON.stringify(parameters)\n   }\n }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        2208,
        3008
      ],
      "id": "2563956e-1d3c-4803-a374-14b3881d3328",
      "name": "Prepare payload21"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n \"statement\": {{ $json.statement }},\n \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        2432,
        3008
      ],
      "id": "bf70a9a1-9c13-4c9d-9f5c-549ced583493",
      "name": "COMMUNICATES_WITH Metrics Update1",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n \"statement\": {{ $json.statement }},\n \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1632,
        3008
      ],
      "id": "4381750a-9c59-435f-9b03-43f4bf24a9d0",

```
      "name": "Get existing relationships",
      "credentials": {
        "httpBasicAuth": {
          "id": "nWJCZ6f1qT459ztQ",
          "name": "neo4j"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const result = $input.first().json;\n\nconst now = new Date();\
nconst decayFactor = 0.95;\n\nlet newDecayed;\nlet newCount30d;\nlet
newTieStrength;\n\nif (!result.data.values || result.data.values.length === 0) {\n\
n  // Relationship does NOT exist\n  newDecayed = 1;\n  newCount30d = 1;\n
newTieStrength = 1;   // First interaction\n\n} else {\n\n  const record =
result.data.values[0];\n\n  const existingCount30d = record[0] || 0;\n  const
existingDecayed = record[1] || 0;\n  const lastSeen = new Date(record[2]);\n\n
const timeDiffDays = (now - lastSeen) / (1000 * 60 * 60 * 24);\n\n  const
decayedValue =\n    existingDecayed * Math.pow(decayFactor, timeDiffDays);\n\n
newDecayed = decayedValue + 1;\n  newCount30d = existingCount30d + 1;\n\n\n
newTieStrength = newDecayed;  \n}\n\nreturn [\n  {\n    json: {\n      newDecayed,\
n      newCount30d,\n      newTieStrength\n    }\n  }\n];\n\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        1904,
        2992
      ],
      "id": "86b47b9e-9450-40d7-854a-acd00508d7f6",
      "name": "Apply Lazy Decay to Past Interactions and Recalculate Tie Strength"
    },
    {
      "parameters": {
        "content": "##  COMMUNICATES_WITH Metrics Update",
        "height": 480,
        "width": 3072,
        "color": 7
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        720,
        2848
      ],
      "typeVersion": 1,
      "id": "ca23c47c-f9f4-4a15-bd92-f6a767dc88a5",
      "name": "Sticky Note19"
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true,
            "leftValue": "",
            "typeValidation": "strict",
            "version": 3
          },
          "conditions": [
```

```
            {
              "id": "a05aff05-5430-453f-a048-43ddeb9d54e5",
              "leftValue": "={{ $json.body.call_status }}",
              "rightValue": "hangup",
              "operator": {
                "type": "string",
                "operation": "notEquals"
              }
            }
          ],
          "combinator": "and"
        },
        "options": {}
      },
      "type": "n8n-nodes-base.if",
      "typeVersion": 2.3,
      "position": [
        384,
        208
      ],
      "id": "44a1c683-de24-4815-b79d-dc299e8c3a1c",
      "name": "Check status of call"
    },
    {
      "parameters": {
        "content": "## Check if call is answered?",
        "height": 384,
        "width": 272,
        "color": 6
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        256,
        64
      ],
      "typeVersion": 1,
      "id": "2add32aa-729c-463b-bd93-690904b4fddf",
      "name": "Sticky Note20"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n  \"statement\": {{ $json.statement }},\n  \"parameters\":
{{ $json.parameters }}\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1408,
        656
      ],
      "id": "c77b4c62-deb6-47ab-a2ac-de441d9c8a24",
      "name": "Caller: CTM Email Match in graph",
```

```
    "credentials": {
      "httpBasicAuth": {
        "id": "nWJCZ6f1qT459ztQ",
        "name": "neo4j"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "content": "## Look up caller in Mautic by phone number",
      "height": 688,
      "width": 3392
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      80,
      928
    ],
    "typeVersion": 1,
    "id": "50757679-85ca-42f1-821c-55055b4ea888",
    "name": "Sticky Note21"
  },
  {
    "parameters": {
      "conditions": {
        "options": {
          "caseSensitive": true,
          "leftValue": "",
          "typeValidation": "strict",
          "version": 3
        },
        "conditions": [
          {
            "id": "388585ec-bebe-4b6c-98f6-be5b941dfe55",
            "leftValue": "={{ $json.total }}",
            "rightValue": "1",
            "operator": {
              "type": "string",
              "operation": "equals"
            }
          }
        ],
        "combinator": "and"
      },
      "options": {}
    },
    "type": "n8n-nodes-base.if",
    "typeVersion": 2.3,
    "position": [
      448,
      1168
    ],
    "id": "184599d0-1227-4b4f-910f-d006860157bb",
    "name": "contact found?"
  },
  {
    "parameters": {
      "url": "https://188375.crm.c1m.ai/api/contacts",
```

```
        "authentication": "predefinedCredentialType",
        "nodeCredentialType": "mauticApi",
        "sendQuery": true,
        "queryParameters": {
          "parameters": [
            {
              "name": "where[0][col]",
              "value": "phone"
            },
            {
              "name": "where[0][expr]",
              "value": "like"
            },
            {
              "name": "where[0][val]",
              "value": "={{ $('Get required
fields').item.json.caller_number_e164 }}"
            }
          ]
        },
        "sendHeaders": true,
        "headerParameters": {
          "parameters": [
            {
              "name": "cookie",
              "value": "mautic_device_id=fy305alfkze5cdufymf6d1n;"
            }
          ]
        },
        "options": {
          "redirect": {
            "redirect": {}
          }
        }
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        224,
        1168
      ],
      "id": "1fcc9ef3-8f5c-4605-b3c0-0171eaf38a27",
      "name": "Search number of caller in mautic2",
      "credentials": {
        "mauticApi": {
          "id": "VdfEZhgJGQuJxwjq",
          "name": "Mautic account 2"
        }
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const inputData = $input.first().json;\nconst data =
Array.isArray(inputData) ? inputData[0] : inputData;\nconst contacts =
data.contacts;\nconst emailSet = new Set();\n\nfor (const contactId in contacts) {\
n  const contact = contacts[contactId];\n\n  const coreEmail =
contact?.fields?.core?.email?.value;\n  if (coreEmail)
emailSet.add(coreEmail.toLowerCase().trim());\n\n  const allEmail =
```

contact?.fields?.all?.email;\n if (allEmail)
emailSet.add(allEmail.toLowerCase().trim());\n\n const notesRaw =
contact?.fields?.core?.contactnotesjson1?.value;\n if (notesRaw) {\n   try {\n
const notes = JSON.parse(notesRaw);\n   if (notes?.data?.work_email)
emailSet.add(notes.data.work_email.toLowerCase().trim());\n   if
(Array.isArray(notes?.data?.personal_emails)) {\n
notes.data.personal_emails.forEach(e => { if (e)
emailSet.add(e.toLowerCase().trim()); });\n   }\n   if
(Array.isArray(notes?.data?.emails)) {\n   notes.data.emails.forEach(e => { if
(e?.address) emailSet.add(e.address.toLowerCase().trim()); });\n   }\n   }
catch (e) {}\n }\n}\n\nconst emails = Array.from(emailSet);\n\nreturn [{ json: { \
n emails: emails,\n contactFound: emails.length > 0\n} }];"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        592,
        1040
      ],
      "id": "11c0fb18-a713-460b-b80b-a190ebee76db",
      "name": "Get email of caller"
    },
    {
      "parameters": {
        "jsCode": "const query = `\nMATCH (phone:PhoneNumber {e164: $callerE164})\
nOPTIONAL MATCH (phone)-[existing:BELONGS_TO {current: true}]-
>(existingPerson:Person)\n\nWITH phone, existing, existingPerson,\n CASE\n   WHEN
existing IS NULL THEN true\n   WHEN $newConfidence > existing.confidence THEN
true\n   ELSE false\n END AS shouldAssign\n\nFOREACH (_ IN CASE WHEN shouldAssign
AND existing IS NOT NULL THEN [1] ELSE [] END |\n SET existing.current = false,\n
existing.validTo = datetime()\n)\n\nWITH phone, shouldAssign\nMATCH (person:Person
{email: $personEmail})\nWHERE shouldAssign\nMERGE (phone)-[b:BELONGS_TO]->(person)\
nON CREATE SET\n b.current = true,\n b.source = $source,\n b.confidence =
$newConfidence,\n b.validFrom = datetime(),\n b.verified = false\n\nRETURN
shouldAssign, person.email\n`;\n\nconst parameters = {\n callerE164: $node['Get
required   fields'].json.callee_number_e164,\n personEmail:
$input.first().json.emails,\n source:'CRM',\n newConfidence:0.95\n \n};\n\
nreturn [\n {\n   json: {\n   statement: JSON.stringify(query),\n
parameters: JSON.stringify(parameters)\n   }\n }\n];\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        944,
        2240
      ],
      "id": "e84f05b7-9d7e-4fb4-b728-7d461bd754aa",
      "name": "Prepare payload18"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://cdfc4331.databases.neo4j.io/db/neo4j/query/v2",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpBasicAuth",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={\n \"statement\": {{ $json.statement }},\n \"parameters\":
{{ $json.parameters }}\n}\n",

```
      "options": {}
    },
    "type": "n8n-nodes-base.httpRequest",
    "typeVersion": 4.3,
    "position": [
      1216,
      2240
    ],
    "id": "7a106fce-2285-46c5-9276-fdb59aca96bf",
    "name": "Create Phone Ownership of Caller4",
    "credentials": {
      "httpBasicAuth": {
        "id": "nWJCZ6f1qT459ztQ",
        "name": "neo4j"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "content": "## Look up callee in Mautic by phone number",
      "height": 736,
      "width": 3392,
      "color": 6
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      128,
      2112
    ],
    "typeVersion": 1,
    "id": "e45c1146-2ae3-474e-850a-a9ff31b7d002",
    "name": "Sticky Note22"
  },
  {
    "parameters": {
      "conditions": {
        "options": {
          "caseSensitive": true,
          "leftValue": "",
          "typeValidation": "strict",
          "version": 3
        },
        "conditions": [
          {
            "id": "388585ec-bebe-4b6c-98f6-be5b941dfe55",
            "leftValue": "={{ $json.total }}",
            "rightValue": "1",
            "operator": {
              "type": "string",
              "operation": "equals"
            }
          }
        ],
        "combinator": "and"
      },
      "options": {}
    },
    "type": "n8n-nodes-base.if",
```

```
      "typeVersion": 2.3,
      "position": [
        544,
        2352
      ],
      "id": "cb33bf26-c5d2-4a1c-9b5c-1eab61df42a6",
      "name": "contact found?1"
    },
    {
      "parameters": {
        "url": "https://188375.crm.c1m.ai/api/contacts",
        "authentication": "predefinedCredentialType",
        "nodeCredentialType": "mauticApi",
        "sendQuery": true,
        "queryParameters": {
          "parameters": [
            {
              "name": "where[0][col]",
              "value": "phone"
            },
            {
              "name": "where[0][expr]",
              "value": "like"
            },
            {
              "name": "where[0][val]",
              "value": "={{ $('Get required
fields').first().json.callee_number_e164 }}"
            }
          ]
        },
        "sendHeaders": true,
        "headerParameters": {
          "parameters": [
            {
              "name": "cookie",
              "value": "mautic_device_id=fy305alfkze5cdufymf6d1n;"
            }
          ]
        },
        "options": {
          "redirect": {
            "redirect": {}
          }
        }
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        288,
        2368
      ],
      "id": "d19d5772-1c68-47ab-b3cd-8a4dc6a5ad30",
      "name": "Search number of callee in mautic",
      "credentials": {
        "mauticApi": {
          "id": "VdfEZhgJGQuJxwjq",
          "name": "Mautic account 2"
        }
```

```
      },
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "jsCode": "const inputData = $input.first().json;\nconst data =
Array.isArray(inputData) ? inputData[0] : inputData;\nconst contacts =
data.contacts;\nconst emailSet = new Set();\n\nfor (const contactId in contacts) {\
n  const contact = contacts[contactId];\n\n  const coreEmail =
contact?.fields?.core?.email?.value;\n  if (coreEmail)
emailSet.add(coreEmail.toLowerCase().trim());\n\n  const allEmail =
contact?.fields?.all?.email;\n  if (allEmail)
emailSet.add(allEmail.toLowerCase().trim());\n\n  const notesRaw =
contact?.fields?.core?.contactnotesjson1?.value;\n  if (notesRaw) {\n    try {\n
const notes = JSON.parse(notesRaw);\n      if (notes?.data?.work_email)
emailSet.add(notes.data.work_email.toLowerCase().trim());\n      if
(Array.isArray(notes?.data?.personal_emails)) {\n
notes.data.personal_emails.forEach(e => { if (e)
emailSet.add(e.toLowerCase().trim()); });\n      }\n      if
(Array.isArray(notes?.data?.emails)) {\n        notes.data.emails.forEach(e => { if
(e?.address) emailSet.add(e.address.toLowerCase().trim()); });\n      }\n    }
catch (e) {}\n  }\n}\n\nconst emails = Array.from(emailSet);\n\nreturn [{ json: { \
n  emails: emails,\n  contactFound: emails.length > 0\n} }];"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        656,
        2240
      ],
      "id": "ede1e06c-0809-4042-a073-0bb4f57b1cb7",
      "name": "Get email of callee"
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "8172d0be-9ecc-475d-8018-6d8318a96ccc",
              "name": "message",
              "value": "call is hanged up",
              "type": "string"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        304,
        -144
      ],
      "id": "15ef64e0-7e9b-4731-8d0e-e474625d038e",
      "name": "Call is not answered"
    },
    {
      "parameters": {
        "content": "## Call is not answered",
```

```
        "height": 288,
        "width": 288,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        256,
        -256
      ],
      "typeVersion": 1,
      "id": "6c0c4727-6a03-4224-b31e-bc93c2f1cccd",
      "name": "Sticky Note1"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://188375.agent.c1m.ai/webhook/fa2d19af-c294-4e6d-ba54-
8c382d90c026",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "chatInput",
              "value": "=Give me detail of {{ $('Webhook').item.json.body.cnam }}
in the form of json"
            },
            {
              "name": "sessionId",
              "value": "={{ $now.toMillis().toString() + '-' +
Math.random().toString(36).substring(2,10) }}"
            }
          ]
        },
        "options": {
          "redirect": {
            "redirect": {}
          }
        }
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.4,
      "position": [
        624,
        1312
      ],
      "id": "79e10b42-06ba-472d-9c24-49254da290e0",
      "name": "Search through Agent-MCPClient-LinkedIn-Automation",
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "model": {
          "__rl": true,
          "mode": "list",
          "value": "gpt-5-mini"
        },
        "builtInTools": {},
        "options": {}
      },
```

```
    "type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
    "typeVersion": 1.3,
    "position": [
      864,
      1488
    ],
    "id": "799800a8-b85e-4d5d-9b87-5e0f3bfc4221",
    "name": "OpenAI Chat Model1",
    "credentials": {
      "openAiApi": {
        "id": "fsHBo55IPrWpeJ8r",
        "name": "OpenAi account"
      }
    }
  },
  {
    "parameters": {
      "authentication": "oAuth2",
      "email": "={{ $json.output.contact_info.work_email }}",
      "firstName": "={{ $json.output.first_name }}",
      "lastName": "={{ $json.output.last_name }}",
      "company": "=",
      "position": "={{ $json.output.current_title }}",
      "title": "={{ $json.output.current_title }}",
      "additionalFields": {
        "mobile": "={{ $json.output.contact_info.mobile }}",
        "phone": "={{ $json.output.contact_info.phone }}",
        "tags": [
          "New - Needs Enrichment"
        ],
        "socialMediaUi": {
          "socialMediaValues": {
            "linkedIn": "={{ $json.output.profile.linkedin_url }}"
          }
        },
        "website": "={{ $json.output.current_company.website }}"
      },
      "options": {
        "rawData": true
      }
    },
    "type": "n8n-nodes-base.mautic",
    "typeVersion": 1,
    "position": [
      1424,
      1264
    ],
    "id": "2d7caa4c-cd35-4571-b95b-367da84f1d8b",
    "name": "Create a contact",
    "credentials": {
      "mauticOAuth2Api": {
        "id": "eo0BF5ZoKECO75Up",
        "name": "Mautic account"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
```

```
      "conditions": {
        "options": {
          "caseSensitive": true,
          "leftValue": "",
          "typeValidation": "strict",
          "version": 3
        },
        "conditions": [
          {
            "id": "cb9a8e27-1415-40b0-abf8-762d6cb9c566",
            "leftValue": "={{ $json.isPublished }}",
            "rightValue": false,
            "operator": {
              "type": "boolean",
              "operation": "true",
              "singleValue": true
            }
          }
        ],
        "combinator": "and"
      },
      "options": {}
    },
    "type": "n8n-nodes-base.if",
    "typeVersion": 2.3,
    "position": [
      1632,
      1312
    ],
    "id": "b3071ca7-3f7f-4c68-86df-a0af1665d686",
    "name": "contact create?"
  },
  {
    "parameters": {
      "assignments": {
        "assignments": [
          {
            "id": "74d1590f-8c1f-46ba-9166-82efe12093a2",
            "name": "status",
            "value": "contact not created in mautic",
            "type": "string"
          }
        ]
      },
      "options": {}
    },
    "type": "n8n-nodes-base.set",
    "typeVersion": 3.4,
    "position": [
      1744,
      1440
    ],
    "id": "e996195a-0c55-413d-9013-402d312e220f",
    "name": "contact not created in mautic"
  },
  {
    "parameters": {
      "method": "POST",
      "url": "https://188375.agent.c1m.ai/webhook/fa2d19af-c294-4e6d-ba54-
```

8c382d90c026",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "chatInput",
              "value": "=Give me detail of {{ $('Get required
fields').first().json.callee_name }} in the form of json"
            },
            {
              "name": "sessionId",
              "value": "={{ $now.toMillis().toString() + '-' +
Math.random().toString(36).substring(2,10) }}"
            }
          ]
        },
        "options": {
          "redirect": {
            "redirect": {}
          }
        }
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.4,
      "position": [
        784,
        2512
      ],
      "id": "1443807c-f3ad-467d-abf5-972c541fcb5e",
      "name": "Search through Agent-MCPClient-LinkedIn-Automation1",
      "onError": "continueErrorOutput"
    },
    {
      "parameters": {
        "model": {
          "__rl": true,
          "mode": "list",
          "value": "gpt-5-mini"
        },
        "builtInTools": {},
        "options": {}
      },
      "type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
      "typeVersion": 1.3,
      "position": [
        1056,
        2720
      ],
      "id": "3420225e-148e-4e7c-afb8-7fc0271619b2",
      "name": "OpenAI Chat Model2",
      "credentials": {
        "openAiApi": {
          "id": "fsHBo55IPrWpeJ8r",
          "name": "OpenAi account"
        }
      },
      "onError": "continueErrorOutput"
    },
    {

```
    "parameters": {
      "authentication": "oAuth2",
      "email": "={{ $json.output.contact_info.work_email }}",
      "firstName": "={{ $json.output.first_name }}",
      "lastName": "={{ $json.output.last_name }}",
      "company": "=",
      "position": "={{ $json.output.current_title }}",
      "title": "=",
      "additionalFields": {
        "mobile": "={{ $json.output.contact_info.mobile }}",
        "phone": "={{ $json.output.contact_info.phone }}",
        "tags": [
          "New - Needs Enrichment"
        ],
        "socialMediaUi": {
          "socialMediaValues": {
            "linkedIn": "={{ $json.output.profile.linkedin_url }}"
          }
        },
        "website": "={{ $json.output.current_company.website }}"
      },
      "options": {
        "rawData": true
      }
    },
    "type": "n8n-nodes-base.mautic",
    "typeVersion": 1,
    "position": [
      1600,
      2512
    ],
    "id": "c204e26c-ab92-41fa-8a0c-537e1d69904a",
    "name": "Create a contact1",
    "credentials": {
      "mauticOAuth2Api": {
        "id": "eo0BF5ZoKECO75Up",
        "name": "Mautic account"
      }
    },
    "onError": "continueErrorOutput"
  },
  {
    "parameters": {
      "conditions": {
        "options": {
          "caseSensitive": true,
          "leftValue": "",
          "typeValidation": "strict",
          "version": 3
        },
        "conditions": [
          {
            "id": "cb9a8e27-1415-40b0-abf8-762d6cb9c566",
            "leftValue": "={{ $json.isPublished }}",
            "rightValue": false,
            "operator": {
              "type": "boolean",
              "operation": "true",
              "singleValue": true
```

```
          }
        }
      ],
      "combinator": "and"
    },
    "options": {}
  },
  "type": "n8n-nodes-base.if",
  "typeVersion": 2.3,
  "position": [
    1840,
    2496
  ],
  "id": "665d1f13-2be3-4a22-880f-4543c3c38716",
  "name": "contact create?1"
},
{
  "parameters": {
    "assignments": {
      "assignments": [
        {
          "id": "74d1590f-8c1f-46ba-9166-82efe12093a2",
          "name": "status",
          "value": "contact not created in mautic",
          "type": "string"
        }
      ]
    },
    "options": {}
  },
  "type": "n8n-nodes-base.set",
  "typeVersion": 3.4,
  "position": [
    2032,
    2608
  ],
  "id": "db2445b8-aa1f-42d9-bb95-a51b3c500053",
  "name": "contact not created in mautic1"
},
{
  "parameters": {
    "assignments": {
      "assignments": [
        {
          "id": "1528f2ab-7d2d-4b82-8dfe-e0a14f8c4ddf",
          "name": "output",
          "value": "={{ $json.output }}",
          "type": "object"
        }
      ]
    },
    "options": {}
  },
  "type": "n8n-nodes-base.set",
  "typeVersion": 3.4,
  "position": [
    1248,
    1312
  ],
```

```
      "id": "321bb145-76e7-40d7-b5bc-4b24e109cf6f",
      "name": "Get output"
    },
    {
      "parameters": {
        "jsonSchemaExample": "{\n  \"output\": {\n    \"first_name\": \"Ashley\",\n
\"last_name\": \"Morris\",\n    \"current_title\": \"Vice President of Project
Management\",\n    \"current_company\": {\n      \"name\": \"Datacubed Health\",\n
\"website\": \"https://datacubed.com\"\n    },\n    \"profile\": {\
n      \"linkedin_url\": \"https://linkedin.com/in/ashley-morris-492081107\"\
n    },\n    \"contact_info\": {\
n      \"work_email\": \"ashley.morris@datacubed.com\",\
n      \"mobile\": \"+14344267675\",\n      \"phone\": \"+14344267675\"\n    }\
n  }\n}"
      },
      "type": "@n8n/n8n-nodes-langchain.outputParserStructured",
      "typeVersion": 1.3,
      "position": [
        1040,
        1488
      ],
      "id": "1410b9ba-bbb0-4a96-a9a2-a41c7ed1c8af",
      "name": "Structured Output Parser"
    },
    {
      "parameters": {
        "jsonSchemaExample": "{\n  \"output\": {\n    \"first_name\": \"Ashley\",\n
\"last_name\": \"Morris\",\n    \"current_title\": \"Vice President of Project
Management\",\n    \"current_company\": {\n      \"name\": \"Datacubed Health\",\n
\"website\": \"https://datacubed.com\"\n    },\n    \"profile\": {\
n      \"linkedin_url\": \"https://linkedin.com/in/ashley-morris-492081107\"\
n    },\n    \"contact_info\": {\
n      \"work_email\": \"ashley.morris@datacubed.com\",\
n      \"mobile\": \"+14344267675\",\n      \"phone\": \"+14344267675\"\n    }\
n  }\n}"
      },
      "type": "@n8n/n8n-nodes-langchain.outputParserStructured",
      "typeVersion": 1.3,
      "position": [
        1232,
        2704
      ],
      "id": "b6bc84ee-f449-4e84-bb0f-3882f4964883",
      "name": "Structured Output Parser1"
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "9ac5c7e9-04be-4f36-91a9-b7927189c8b2",
              "name": "value1",
              "value": "={{ $('Get
output').item.json.output.contact_info.work_email }}",
              "type": "string"
            },
            {
              "id": "c8bbb685-bebc-444c-84f4-d1eb8f6dc9f6",
              "name": "value2",
```

```
              "value": "={{ $('get
output1').item.json.output.contact_info.work_email }}",
              "type": "string"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        1024,
        3008
      ],
      "id": "7abc6bcc-c468-41cf-8fba-f07d7aaff02d",
      "name": "Set values"
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "1528f2ab-7d2d-4b82-8dfe-e0a14f8c4ddf",
              "name": "output",
              "value": "={{ $json.output }}",
              "type": "object"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        1408,
        2496
      ],
      "id": "b0908242-3372-4f2b-b9f0-9ae35312b24b",
      "name": "get output1"
    },
    {
      "parameters": {
        "mode": "raw",
        "jsonOutput": "{\n  \"message\": \"no further action\"\n\n}\n",
        "options": {}
      },
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        352,
        3568
      ],
      "id": "086327cb-68c0-4da1-a655-889f4f881192",
      "name": "call donot qualify for RAG"
    },
    {
      "parameters": {
        "promptType": "define",
        "text": "={{ $json.output }}",
```

"hasOutputParser": true,
"options": {
"systemMessage": "You are a JSON repair specialist.\n\nYour task is to receive a broken or invalid JSON string and repair it so it becomes perfectly valid JSON.\n\nRules:\n- Remove all JavaScript-style comments (// and /* */)\n- Remove trailing commas before } or]\n- Remove any text appearing after the final closing brace }\n- Fix invalid control characters in strings\n- Fix unescaped special characters inside strings\n- Preserve all existing keys and values exactly as they appear\n- Do not invent new fields or modify existing data\n- Do not remove fields unless they make the JSON invalid\n- Return only valid JSON\n\nImportant:\nThe final JSON must strictly follow the structure required by the Structured Output Parser.\n\nOutput ONLY the raw JSON object.\nDo not include explanations or markdown formatting."
}
},
"type": "@n8n/n8n-nodes-langchain.agent",
"typeVersion": 3.1,
"position": [
896,
1280
],
"id": "40937794-9bcb-4997-98ad-ebfb02218b91",
"name": "JSON Parser",
"onError": "continueErrorOutput"
},
{
"parameters": {
"promptType": "define",
"text": "={{ $json.output }}",
"hasOutputParser": true,
"options": {
"systemMessage": "You are a JSON repair specialist.\n\nYour task is to receive a broken or invalid JSON string and repair it so it becomes perfectly valid JSON.\n\nRules:\n- Remove all JavaScript-style comments (// and /* */)\n- Remove trailing commas before } or]\n- Remove any text appearing after the final closing brace }\n- Fix invalid control characters in strings\n- Fix unescaped special characters inside strings\n- Preserve all existing keys and values exactly as they appear\n- Do not invent new fields or modify existing data\n- Do not remove fields unless they make the JSON invalid\n- Return only valid JSON\n\nImportant:\nThe final JSON must strictly follow the structure required by the Structured Output Parser.\n\nOutput ONLY the raw JSON object.\nDo not include explanations or markdown formatting."
}
},
"type": "@n8n/n8n-nodes-langchain.agent",
"typeVersion": 3.1,
"position": [
1056,
2512
],
"id": "f9093d42-d462-481a-be94-f74194ead58c",
"name": "JSON PARSER",
"onError": "continueErrorOutput"
},
{
"parameters": {
"content": "## Getting lead information",
"height": 384,
"width": 784,

```
      "color": 6
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      592,
      1216
    ],
    "typeVersion": 1,
    "id": "560af920-1022-4c7e-a80e-78f3b3ac0cde",
    "name": "Sticky Note23"
  },
  {
    "parameters": {
      "content": "## Creating contact in mautic",
      "height": 384,
      "width": 544,
      "color": 7
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      1392,
      1216
    ],
    "typeVersion": 1,
    "id": "012e753c-71ab-4fb4-aecf-d327a8fbf56b",
    "name": "Sticky Note24"
  },
  {
    "parameters": {
      "content": "## Getting lead information",
      "height": 384,
      "width": 784,
      "color": 5
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      720,
      2432
    ],
    "typeVersion": 1,
    "id": "e348da7b-de6d-4b5f-8c6d-63507b6991ec",
    "name": "Sticky Note25"
  },
  {
    "parameters": {
      "content": "## Creating contact in mautic",
      "height": 384,
      "width": 416,
      "color": 7
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      1536,
      2432
    ],
    "typeVersion": 1,
    "id": "d97e3717-68c2-4c1b-81d0-6a8bfd5eac84",
    "name": "Sticky Note26"
  },
```

```
    {
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
executionDatetime: new Date().getTime(),\n  executionId: $execution.id,\n
executionState: \"Start\",\n  data: JSON.stringify({\n  type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n  metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        -224,
        -240
      ],
      "id": "e6a9c39b-8776-4e37-9f16-7bad309f7d17",
      "name": "Prepare payload for TOOL-LogEvent"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://admin.n8n.c1m.ai/webhook/7fd5766d-c2de-47c5-8e5a-
df80535a7ad3",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "data",
              "value": "={{ $json }}"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        32,
        -240
      ],
      "id": "7c671d3b-8492-4942-aefd-50c1693c5b3d",
      "name": "Invoke TOOL-LogEvent",
      "alwaysOutputData": true
    },
    {
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
executionDatetime: new Date().getTime(),\n  executionId: $execution.id,\n
executionState: \"Complete\",\n  data: JSON.stringify({\n  type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n  metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        960,
        3968
      ],
      "id": "532bb753-1771-429e-9446-72acff4f3c8d",
      "name": "Prepare payload for TOOL-LogEvent1"
```

```
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://admin.n8n.c1m.ai/webhook/7fd5766d-c2de-47c5-8e5a-
df80535a7ad3",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "data",
              "value": "={{ $json }}"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        1328,
        4080
      ],
      "id": "b80e0278-97b9-4612-878b-b02bf6aac6f1",
      "name": "Invoke TOOL-LogEvent1",
      "alwaysOutputData": true
    },
    {
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
executionDatetime: new Date().getTime(),\n  executionId: $execution.id,\n
executionState: \"Error\",\n  data: JSON.stringify({\n  type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n  metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        960,
        4208
      ],
      "id": "2cf4415e-f854-41a0-b6e7-5a5e01dfe3fb",
      "name": "Prepare payload for TOOL-LogEvent2"
    },
    {
      "parameters": {
        "content": "## Payload of success",
        "height": 224,
        "width": 336,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        864,
        3904
      ],
      "typeVersion": 1,
      "id": "bcecf668-6792-4991-a851-2c590e57bdb9",
      "name": "Sticky Note18"
```

```
    },
    {
      "parameters": {
        "content": "## Payload of error",
        "height": 208,
        "width": 352,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        864,
        4144
      ],
      "typeVersion": 1,
      "id": "298dff9a-34d1-4f9d-b6f6-79479e2a5ac0",
      "name": "Sticky Note27"
    },
    {
      "parameters": {
        "content": "## Logging",
        "height": 224,
        "width": 272
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        1248,
        4032
      ],
      "typeVersion": 1,
      "id": "9d5c7342-4d70-4234-b92c-aa93d606aea1",
      "name": "Sticky Note28"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://admin.n8n.c1m.ai/webhook/7fd5766d-c2de-47c5-8e5a-
df80535a7ad3",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "data",
              "value": "={{ $json }}"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        -240,
        4080
      ],
      "id": "6779e610-f6ac-4cea-8174-a34f39524856",
      "name": "Invoke TOOL-LogEvent2",
      "alwaysOutputData": true
    },
    {
```

```
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
executionDatetime: new Date().getTime(),\n  executionId: $execution.id,\n
executionState: \"Error\",\n  data: JSON.stringify({\n  type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n  metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        -608,
        4080
      ],
      "id": "8921d7c4-80ba-45f4-8c05-57bd8a73fc46",
      "name": "Prepare payload for TOOL-LogEvent4"
    },
    {
      "parameters": {
        "content": "## Logging",
        "height": 224,
        "width": 272
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -336,
        4016
      ],
      "typeVersion": 1,
      "id": "354d158a-c684-4d3c-8f67-f90b52360ac1",
      "name": "Sticky Note29"
    },
    {
      "parameters": {
        "content": "## Payload of error",
        "height": 208,
        "width": 352,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -720,
        4032
      ],
      "typeVersion": 1,
      "id": "c1cee3db-6a75-4f5c-92db-97c1792388bf",
      "name": "Sticky Note30"
    },
    {
      "parameters": {
        "numberInputs": 10
      },
      "type": "n8n-nodes-base.merge",
      "typeVersion": 3.2,
      "position": [
        -944,
        3952
      ],
      "id": "dd0d5846-6848-4e2d-96d8-70138e196db9",
      "name": "Combine all errors"
```

```
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://admin.n8n.c1m.ai/webhook/7fd5766d-c2de-47c5-8e5a-
df80535a7ad3",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "data",
              "value": "={{ $json }}"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        -160,
        2400
      ],
      "id": "deaf89b2-eab0-4287-a083-6170964a4199",
      "name": "Invoke TOOL-LogEvent3",
      "alwaysOutputData": true
    },
    {
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
executionDatetime: new Date().getTime(),\n  executionId: $execution.id,\n
executionState: \"Error\",\n  data: JSON.stringify({\n  type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n  metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        -528,
        2400
      ],
      "id": "43e3d8bd-c99f-427e-a449-3281d9f0073e",
      "name": "Prepare payload for TOOL-LogEvent5"
    },
    {
      "parameters": {
        "content": "## Logging",
        "height": 224,
        "width": 272
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -256,
        2336
      ],
      "typeVersion": 1,
      "id": "dd22e548-3caa-44a9-82cf-c6bc8ce8f08e",
      "name": "Sticky Note31"
    },
```

```
    {
      "parameters": {
        "content": "## Payload of error",
        "height": 208,
        "width": 352,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -640,
        2352
      ],
      "typeVersion": 1,
      "id": "bba655e5-efa1-4d8b-9d56-b56f63913710",
      "name": "Sticky Note32"
    },
    {
      "parameters": {
        "numberInputs": 10
      },
      "type": "n8n-nodes-base.merge",
      "typeVersion": 3.2,
      "position": [
        -864,
        2272
      ],
      "id": "4d157a95-871d-4cac-879b-ac851850206c",
      "name": "Combine all errors1"
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://admin.n8n.c1m.ai/webhook/7fd5766d-c2de-47c5-8e5a-
df80535a7ad3",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "data",
              "value": "={{ $json }}"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        -192,
        1408
      ],
      "id": "70bf9e29-f9fd-4cc0-82b5-5cdc39097cf2",
      "name": "Invoke TOOL-LogEvent4",
      "alwaysOutputData": true
    },
    {
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
```

executionDatetime: new Date().getTime(),\n executionId: $execution.id,\n
executionState: \"Error\",\n data: JSON.stringify({\n type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        -560,
        1408
      ],
      "id": "ca7ee0b8-a8bf-4369-a815-23eac14a93cc",
      "name": "Prepare payload for TOOL-LogEvent6"
    },
    {
      "parameters": {
        "content": "## Logging",
        "height": 224,
        "width": 272
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -288,
        1344
      ],
      "typeVersion": 1,
      "id": "6f473942-6a91-459c-aa4c-06a15f18d626",
      "name": "Sticky Note33"
    },
    {
      "parameters": {
        "content": "## Payload of error",
        "height": 208,
        "width": 352,
        "color": 5
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -672,
        1360
      ],
      "typeVersion": 1,
      "id": "005ad91a-c3cd-4ed4-980a-02b31fb29ce5",
      "name": "Sticky Note34"
    },
    {
      "parameters": {
        "numberInputs": 10
      },
      "type": "n8n-nodes-base.merge",
      "typeVersion": 3.2,
      "position": [
        -896,
        1280
      ],
      "id": "40a54ef9-a97f-486b-8fc5-4dea6a8019ed",
      "name": "Combine all errors2"
    },
    {
      "parameters": {

```
        "method": "POST",
        "url": "https://admin.n8n.c1m.ai/webhook/7fd5766d-c2de-47c5-8e5a-
df80535a7ad3",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "data",
              "value": "={{ $json }}"
            }
          ]
        },
        "options": {}
      },
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.3,
      "position": [
        -544,
        128
      ],
      "id": "6e6c2acb-d169-451b-880a-42a8e46ac189",
      "name": "Invoke TOOL-LogEvent5",
      "alwaysOutputData": true
    },
    {
      "parameters": {
        "jsCode": "return {\n  propertyId: 188375,\n  serviceId: $workflow.id,\n
serviceName: \"Tool-Process-CallRecordings-Relationships-Webhook\",\n
executionDatetime: new Date().getTime(),\n  executionId: $execution.id,\n
executionState: \"Error\",\n  data: JSON.stringify({\n  type: \"webhook\",\n
content: $input.first().json.body\n}),\n\n  metadata: $execution.customData,\n}\n"
      },
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [
        -912,
        128
      ],
      "id": "3cafef31-dceb-4c1e-88f5-c512c9cd65dc",
      "name": "Prepare payload for TOOL-LogEvent7"
    },
    {
      "parameters": {
        "content": "## Logging",
        "height": 224,
        "width": 272
      },
      "type": "n8n-nodes-base.stickyNote",
      "position": [
        -640,
        64
      ],
      "typeVersion": 1,
      "id": "b26fb5fe-234b-4dce-bb2c-60cf39b2e2cb",
      "name": "Sticky Note35"
    },
    {
      "parameters": {
        "content": "## Payload of error",
```

```
      "height": 208,
      "width": 352,
      "color": 5
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      -1024,
      80
    ],
    "typeVersion": 1,
    "id": "9f71d0d8-058d-4e2e-80ed-8a0191d6eae0",
    "name": "Sticky Note36"
  },
  {
    "parameters": {
      "numberInputs": 10
    },
    "type": "n8n-nodes-base.merge",
    "typeVersion": 3.2,
    "position": [
      -1248,
      0
    ],
    "id": "1f305be1-478a-48e1-9ce8-be7428c1a04e",
    "name": "Combine all errors3"
  },
  {
    "parameters": {
      "content": "## Logging start",
      "height": 304,
      "width": 608,
      "color": 6
    },
    "type": "n8n-nodes-base.stickyNote",
    "position": [
      -384,
      -304
    ],
    "typeVersion": 1,
    "id": "c492dc80-7f8d-401e-b283-de2f6a3dacf6",
    "name": "Sticky Note37"
  }
],
"pinData": {
  "Webhook": [
    {
      "json": {
        "headers": {
          "connection": "close",
          "host": "n8n.c1m.ai",
          "x-real-ip": "52.204.198.9",
          "x-forwarded-for": "52.204.198.9",
          "x-forwarded-proto": "https",
          "content-length": "6496",
          "x-ctm-time": "1772557846",
          "x-ctm-webhook": "261038 end",
          "content-type": "application/json",
          "accept": "application/json",
          "user-agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5)
```

AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36",
            "x-ctm-signature": "XW+xJC8joYIewwZma4fcgRk6l6Q="
          },
          "params": {},
          "query": {},
          "body": {
            "id": 4059685916,
            "sid": "CA3d966c54ba2fce4bffa7693890623133",
            "account_id": 210737,
            "name": "Ashley Pp",
            "cnam": "Ashley Morris",
            "search": "",
            "referrer": "",
            "visitor_sid": "",
            "location": "",
            "source": "Email",
            "source_id": 1002831,
            "source_sid":
"TSOF6FC2D2594D22C52BC4BD29AE71572E9D961C64EEE7B1001674FB79108853308",
            "tgid": 3134604,
            "likelihood": null,
            "duration": 147,
            "direction": "inbound",
            "talk_time": 135,
            "ring_time": 13,
            "hold_time": 0,
            "wait_time": 0,
            "parent_id": null,
            "email": null,
            "street": "",
            "city": "Jackson",
            "state": "MS",
            "country": null,
            "postal_code": "39201",
            "called_at": "2026-03-03 11:07 AM -06:00",
            "unix_time": 1772557647,
            "tracking_number_id": 1904826,
            "tracking_number_sid":
"TPNC3C4B23C348AEC2EE54EFD301979CD2E4D68FDEF6CEB425F400735BD54E9F9C718D51A4144350B4
4",
            "tracking_number": "+12142252647",
            "tracking_label": "Laurie Lindloff Cell",
            "dial_status": "completed",
            "is_new_caller": false,
            "indexed_at": 1772557845.0918808,
            "agent_insights": false,
            "duration_period": {
              "gte": 1772557647000,
              "lte": 1772557794000
            },
            "inbound_rate_center": {
              "country": "US",
              "prefix": "1",
              "tollfree": false
            },
            "business_number": "+17604921465",
            "business_label": null,
            "receiving_number_id": 1285572,
            "receiving_number_sid":

"RPN34D8AC3F61E8848FEA641CEF711011AA7D7B791FA7F7AD9A6E8D338CA1BC98F20C5C69B2B9FDF4D
E",
            "billed_amount": 1200,
            "billed_at": "2026-03-03T17:09:56",
            "excluded": false,
            "redacted": false,
            "tracking_number_format": "(214) 225-2647",
            "business_number_format": "(760) 492-1465",
            "alternative_number": "6016240654",
            "tracking_number_bare": "2142252647",
            "caller_number_complete": "+16016240654",
            "caller_number_format": "(601) 624-0654",
            "caller_number_bare": "6016240654",
            "caller_number": "+16016240654",
            "caller_number_split": [
              "1",
              "601",
              "624",
              "0654"
            ],
            "contact_number": "+16016240654",
            "visitor": false,
            "call_path": [
              {
                "route_name": " (760) 492-1465",
                "route_id":
"RPN34D8AC3F61E8848FEA641CEF711011AA7D7B791FA7F7AD9A6E8D338CA1BC98F20C5C69B2B9FDF4D
E",
                "route_type": "PhysicalPhoneNumber",
                "started_at": "2026-03-03T11:07:27Z"
              }
            ],
            "left_talk_time": 112,
            "right_talk_time": 102,
            "transfers": [],
            "call_status": "answered",
            "status": "answered",
            "spotted": [],
            "audio":
"https://app.calltrackingmetrics.com/api/v1/accounts/210737/calls/
CA3d966c54ba2fce4bffa7693890623133/recording",
            "is_s3_link": true,
            "transcription":
"/api/v1/accounts/210737/calls/4059685916/transcription.json",
            "transcription_text": "+17604921465: Hi, Ashley.\nAshley Pp: Hi, Lori.\
n+17604921465: Hey. So we're just testing I I forgot what the purpose of the of the
call\nAshley Pp: We're just\n+17604921465: was.\nAshley Pp: supposed to stay on the
call for\n+17604921465: Okay.\nAshley Pp: one minute to make sure it registers for,
Nabil and Vicus. I think that's their rule. It doesn't count unless it's over a
minute.\n+17604921465: Okay.\nAshley Pp: Yep. So\n+17604921465: Okay.\nAshley Pp:
we\n+17604921465: Well,\nAshley Pp: can just hang up after that. Go ahead. Yeah. I
was out, getting my walk, so it was perfect. I was just walking back in.\
n+17604921465: oh, good for you. Yeah. I need to be better about that. Getting out\
nAshley Pp: So\n+17604921465: and getting\nAshley Pp: my\n+17604921465: your\
nAshley Pp: friend\n+17604921465: life\nAshley Pp: made\n+17604921465: on.\nAshley
Pp: this calendar, and it's called Mals for March, where you're supposed to get a
mile a day. And if you miss a day, you just make it up the next day.\n+17604921465:
Yeah.\nAshley Pp: But you know what? It's been such a nice reminder for me to just
get out and get a mile in.\n+17604921465: Yeah.\nAshley Pp: And, you know, most

times you go over anyway. Like,\n+17604921465: Yeah. Yeah.\nAshley Pp: just yeah.\n+17604921465: Is it,\nAshley Pp: That\n+17604921465: like,\nAshley Pp: has\n+17604921465: about\nAshley Pp: been and we're both\n+17604921465: twenty minutes?\nAshley Pp: what'd\n+17604921465: It's\nAshley Pp: you say?\n+17604921465: about a mile. It's twenty\nAshley Pp: Yeah.\n+17604921465: minutes, fifteen minutes. Yeah. Fifteen, twenty minutes.\nAshley Pp: Yes.\n+17604921465: Yeah. Yeah.\nAshley Pp: Yep. Well, it's still taking about 20. I haven't built back up to my 15 yet, but at least\n+17604921465: Yeah. No. It takes me about 20. Yeah. About the 18\nAshley Pp: Yeah.\n+17604921465: ish.\nAshley Pp: And and\n+17604921465: Yeah.\nAshley Pp: she just had a baby too, so it's just a nice way for us to get moving.\n+17604921465: Oh, that's good. Well, good.\nAshley Pp: Yeah.\n+17604921465: Good for\nAshley Pp: But\n+17604921465: you.\nAshley Pp: you know we have, like, nine days of rain coming. Did you see that?\n+17604921465: I saw my well, my husband told me something about it, so I was like, oh, no. Well, we'll\nAshley Pp: Yeah.\n+17604921465: see.\nAshley Pp: I was on\n+17604921465: Sometimes\nAshley Pp: I was on chat. I was trying to figure out, do I need to plant my flowers now or after? But said after.\n+17604921465: Yeah.\nAshley Pp: So it wouldn't stress out the seed. It said\n+17604921465: Oh, okay.\nAshley Pp: let the soil be wet. But, yeah, I\n+17604921465: Yeah.\nAshley Pp: was saying with that much rain and Texas normally not getting that much, do\n+17604921465: Yeah.\nAshley Pp: it right after.\n+17604921465: Yeah. It might just wash it out or yeah.\nAshley Pp: Yeah.\n+17604921465: Okay. Well, that's good to know.\nAshley Pp: Yep.\n+17604921465: Alright.\nAshley Pp: So I'll wait.\n+17604921465: Yeah.\nAshley Pp: Well,\n+17604921465: Just\nAshley Pp: I think\n+17604921465: wait.\nAshley Pp: we've hit our minute mark. So\n+17604921465: Yeah. Well,\nAshley Pp: because\n+17604921465: good\nAshley Pp: you know\n+17604921465: talking\nAshley Pp: how to\n+17604921465: with you, Ashley. I hope you have a good day.\nAshley Pp: you as well. Talk later.\n+17604921465: Alrighty. Bye bye.\nAshley Pp: K.",
"custom_fields": {},
"callbacks": [],
"emails": [],
"day": "Tuesday",
"month": "March",
"hour": "11AM",
"paid": {
"source": null,
"medium": null
},
"ga": {
"cid": "210737.4059685916",
"ga4": []
},
"tag_list": [],
"notes": "",
"latitude": 32.3113,
"longitude": -90.3972,
"extended_lookup_on": false,
"extended_lookup": {
"first_name": null,
"last_name": null,
"name_type": "B",
"phone_type": "mobile",
"business_name": "Ashley Pp",
"street_number": null,
"street_name": null,
"gender": null,
"city": "Jackson",
"state": "MS",
"zipcode": "39201",

```
              "facebook": null,
              "linkedin": null,
              "twitter": null,
              "education": null,
              "language": null,
              "carrier": null,
              "age": null,
              "high_net_worth": null,
              "home_owner_status": null,
              "household_income": null,
              "marital_status": null,
              "length_of_residence": null,
              "market_value": null,
              "occupation": null,
              "presence_of_children": null
            },
            "legs": []
          },
          "webhookUrl": "https://n8n.c1m.ai/webhook/5d5fecbb-64b8-42d0-b5c6-
30a9e691d959",
          "executionMode": "production"
        }
      }
    ]
  },
  "connections": {
    "Webhook": {
      "main": [
        [
          {
            "node": "Check status of call",
            "type": "main",
            "index": 0
          },
          {
            "node": "Prepare payload for TOOL-LogEvent",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Get required fields": {
      "main": [
        [
          {
            "node": "Prepare payload",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Prepare payload": {
      "main": [
        [
          {
            "node": "Get Call node details",
            "type": "main",
```

```
        "index": 0
      }
    ]
  ]
},
"Get Call node details": {
  "main": [
    [
      {
        "node": "Check if call node already exists1",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors3",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Check if call node already exists1": {
  "main": [
    [
      {
        "node": "Prepare payload2",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Prepare payload1",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload1": {
  "main": [
    [
      {
        "node": "Create call node",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload2": {
  "main": [
    [
      {
        "node": "Caller PhoneNumber",
        "type": "main",
        "index": 0
```

```
      }
    ]
  ]
},
"Create call node": {
  "main": [
    [
      {
        "node": "Prepare payload2",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors3",
        "type": "main",
        "index": 1
      }
    ]
  ]
},
"Prepare payload3": {
  "main": [
    [
      {
        "node": "Callee PhoneNumber",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Caller PhoneNumber": {
  "main": [
    [
      {
        "node": "Prepare payload3",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors3",
        "type": "main",
        "index": 2
      }
    ]
  ]
},
"Prepare payload4": {
  "main": [
    [
      {
        "node": "Create ORIGINATED Relationship",
        "type": "main",
        "index": 0
      }
```

```
      ]
    ]
  },
  "Prepare payload5": {
    "main": [
      [
        {
          "node": "Create REACHED Relationship",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Callee PhoneNumber": {
    "main": [
      [
        {
          "node": "Prepare payload4",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors3",
          "type": "main",
          "index": 3
        }
      ]
    ]
  },
  "Create ORIGINATED Relationship": {
    "main": [
      [
        {
          "node": "Prepare payload5",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors3",
          "type": "main",
          "index": 4
        }
      ]
    ]
  },
  "Prepare payload6": {
    "main": [
      [
        {
          "node": "Check for a Direct Phone-to-Person Match",
          "type": "main",
          "index": 0
        }
      ]
```

```
    ]
  },
  "Create REACHED Relationship": {
    "main": [
      [
        {
          "node": "Prepare payload6",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors3",
          "type": "main",
          "index": 6
        }
      ]
    ]
  },
  "Check for a Direct Phone-to-Person Match": {
    "main": [
      [
        {
          "node": "Caller Direct Match?",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors3",
          "type": "main",
          "index": 5
        }
      ]
    ]
  },
  "Caller Direct Match?": {
    "main": [
      [
        {
          "node": "Prepare payload13",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Has CTM Email?",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Has CTM Email?": {
    "main": [
      [
```

```
      {
        "node": "Prepare payload7",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Search number of caller in mautic2",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload7": {
  "main": [
    [
      {
        "node": "Caller: CTM Email Match in graph",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"If": {
  "main": [
    [
      {
        "node": "Prepare payload14",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Search number of caller in mautic2",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload8": {
  "main": [
    [
      {
        "node": "Link Caller to Call",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload9": {
  "main": [
    [
      {
```

```
        "node": "Check for a Direct Phone-to-Person Match1",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Check for a Direct Phone-to-Person Match1": {
  "main": [
    [
      {
        "node": "Caller Direct Match?1",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
        "index": 5
      }
    ]
  ]
},
"Caller Direct Match?1": {
  "main": [
    [
      {
        "node": "Prepare payload16",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Has CTM Email?1",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Has CTM Email?1": {
  "main": [
    [
      {
        "node": "Prepare payload10",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Search number of callee in mautic",
        "type": "main",
        "index": 0
      }
    ]
```

```
    ]
  },
  "Prepare payload10": {
    "main": [
      [
        {
          "node": "Caller: CTM Email Match2",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Caller: CTM Email Match2": {
    "main": [
      [
        {
          "node": "If1",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors2",
          "type": "main",
          "index": 7
        }
      ]
    ]
  },
  "If1": {
    "main": [
      [
        {
          "node": "Prepare payload17",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Search number of callee in mautic",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Link Caller to Call": {
    "main": [
      [
        {
          "node": "Prepare payload9",
          "type": "main",
          "index": 0
        }
      ],
      [
```

```
      {
        "node": "Combine all errors3",
        "type": "main",
        "index": 9
      }
    ]
  ]
},
"Prepare payload12": {
  "main": [
    [
      {
        "node": "Link Callee to Call",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Merge": {
  "main": [
    [
      {
        "node": "Prepare payload8",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Link Callee to Call": {
  "main": [
    [
      {
        "node": "Qualify for metrics?",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
        "index": 9
      }
    ]
  ]
},
"Merge1": {
  "main": [
    [
      {
        "node": "Prepare payload12",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
```

```
"Qualify for RAG?": {
  "main": [
    [
      {
        "node": "if transcript found?",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "call donot qualify for RAG",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Qualify for metrics?": {
  "main": [
    [
      {
        "node": "Set values",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "status",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"if transcript found?": {
  "main": [
    [
      {
        "node": "Redact Sensitive Information Using Pattern Matching",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Redact Sensitive Information Using Pattern Matching": {
  "main": [
    [
      {
        "node": "Redact Additional PII Using AI-Based Recognition",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Redact Additional PII Using AI-Based Recognition": {
```

```
    "main": [
      [
        {
          "node": "AI Summary & Sales Intelligence Extraction",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "OpenAI Chat Model": {
    "ai_languageModel": [
      [
        {
          "node": "AI Summary & Sales Intelligence Extraction",
          "type": "ai_languageModel",
          "index": 0
        }
      ]
    ],
    "main": [
      [
        {
          "node": "Combine all errors",
          "type": "main",
          "index": 2
        }
      ]
    ]
  },
  "AI Summary & Sales Intelligence Extraction": {
    "main": [
      [
        {
          "node": "Parse Json",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors",
          "type": "main",
          "index": 1
        }
      ]
    ]
  },
  "Default Data Loader": {
    "ai_document": [
      [
        {
```

```
        "node": "Insert transcript into calltranscript db",
        "type": "ai_document",
        "index": 0
      }
    ]
  ],
  "main": [
    [
      {
        "node": "Combine all errors",
        "type": "main",
        "index": 3
      }
    ]
  ]
},
"Recursive Character Text Splitter": {
  "ai_textSplitter": [
    [
      {
        "node": "Default Data Loader",
        "type": "ai_textSplitter",
        "index": 0
      }
    ]
  ],
  "main": [
    [
      {
        "node": "Combine all errors",
        "type": "main",
        "index": 4
      }
    ]
  ]
},
"Prepare payload13": {
  "main": [
    [
      {
        "node": "Create Phone Ownership of Caller",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Create Phone Ownership of Caller": {
  "main": [
    [
      {
        "node": "Merge",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors3",
```

```
        "type": "main",
        "index": 7
      }
    ]
  ]
},
"Prepare payload14": {
  "main": [
    [
      {
        "node": "Create Phone Ownership of Caller1",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload15": {
  "main": [
    [
      {
        "node": "Create Phone Ownership of Caller2",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Create Phone Ownership of Caller1": {
  "main": [
    [
      {
        "node": "Merge",
        "type": "main",
        "index": 1
      }
    ],
    [
      {
        "node": "Combine all errors3",
        "type": "main",
        "index": 9
      }
    ]
  ]
},
"Create Phone Ownership of Caller2": {
  "main": [
    [
      {
        "node": "Merge",
        "type": "main",
        "index": 2
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
```

```
        "index": 0
      }
    ]
  ]
},
"Prepare payload16": {
  "main": [
    [
      {
        "node": "Create Phone Ownership of Callee",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Create Phone Ownership of Callee": {
  "main": [
    [
      {
        "node": "Merge1",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
        "index": 6
      }
    ]
  ]
},
"Prepare payload17": {
  "main": [
    [
      {
        "node": "Create Phone Ownership of Caller3",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Create Phone Ownership of Caller3": {
  "main": [
    [
      {
        "node": "Merge1",
        "type": "main",
        "index": 1
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
        "index": 8
```

```
      }
    ]
  ]
},
"Prepare payload19": {
  "main": [
    [
      {
        "node": "Update RAG References on Call node",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Get transcript": {
  "main": [
    [
      {
        "node": "Insert transcript into calltranscript db",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Embeddings OpenAI": {
  "ai_embedding": [
    [
      {
        "node": "Insert transcript into calltranscript db",
        "type": "ai_embedding",
        "index": 0
      }
    ]
  ],
  "main": [
    [
      {
        "node": "Combine all errors",
        "type": "main",
        "index": 5
      }
    ]
  ]
},
"Parse Json": {
  "main": [
    [
      {
        "node": "Get transcript",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Insert transcript into calltranscript db": {
  "main": [
```

```
      [
        {
          "node": "Get summary",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Default Data Loader1": {
    "ai_document": [
      [
        {
          "node": "Insert summary into summary db",
          "type": "ai_document",
          "index": 0
        }
      ]
    ],
    "main": [
      [
        {
          "node": "Combine all errors",
          "type": "main",
          "index": 6
        }
      ]
    ]
  },
  "Recursive Character Text Splitter1": {
    "ai_textSplitter": [
      [
        {
          "node": "Default Data Loader1",
          "type": "ai_textSplitter",
          "index": 0
        }
      ]
    ],
    "main": [
      [
        {
          "node": "Combine all errors",
          "type": "main",
          "index": 8
        }
      ]
    ]
  },
  "Embeddings OpenAI1": {
    "ai_embedding": [
      [
        {
          "node": "Insert summary into summary db",
          "type": "ai_embedding",
          "index": 0
        }
      ]
    ],
```

```
    "main": [
      [
        {
          "node": "Combine all errors",
          "type": "main",
          "index": 9
        }
      ]
    ]
  },
  "Get summary": {
    "main": [
      [
        {
          "node": "Insert summary into summary db",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Insert summary into summary db": {
    "main": [
      [
        {
          "node": "Set transcript and summary ids",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Set transcript and summary ids": {
    "main": [
      [
        {
          "node": "Prepare payload19",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Prepare payload20": {
    "main": [
      [
        {
          "node": "Get existing relationships",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Prepare payload21": {
    "main": [
      [
        {
          "node": "COMMUNICATES_WITH Metrics Update1",
```

```
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Get existing relationships": {
      "main": [
        [
          {
            "node": "Apply Lazy Decay to Past Interactions and Recalculate Tie
Strength",
            "type": "main",
            "index": 0
          }
        ],
        [
          {
            "node": "Combine all errors1",
            "type": "main",
            "index": 7
          }
        ]
      ]
    },
    "Apply Lazy Decay to Past Interactions and Recalculate Tie Strength": {
      "main": [
        [
          {
            "node": "Prepare payload21",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Check status of call": {
      "main": [
        [
          {
            "node": "Get required fields",
            "type": "main",
            "index": 0
          }
        ],
        [
          {
            "node": "Call is not answered",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Caller: CTM Email Match in graph": {
      "main": [
        [
          {
            "node": "If",
```

```
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors3",
          "type": "main",
          "index": 8
        }
      ]
    ]
  },
  "Search number of caller in mautic2": {
    "main": [
      [
        {
          "node": "contact found?",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors2",
          "type": "main",
          "index": 1
        }
      ]
    ]
  },
  "contact found?": {
    "main": [
      [
        {
          "node": "Get email of caller",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Search through Agent-MCPClient-LinkedIn-Automation",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Get email of caller": {
    "main": [
      [
        {
          "node": "Prepare payload15",
          "type": "main",
          "index": 0
        }
      ]
    ]
```

```
    },
    "Prepare payload18": {
      "main": [
        [
          {
            "node": "Create Phone Ownership of Caller4",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "contact found?1": {
      "main": [
        [
          {
            "node": "Get email of callee",
            "type": "main",
            "index": 0
          }
        ],
        [
          {
            "node": "Search through Agent-MCPClient-LinkedIn-Automation1",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Search number of callee in mautic": {
      "main": [
        [
          {
            "node": "contact found?1",
            "type": "main",
            "index": 0
          }
        ],
        [
          {
            "node": "Combine all errors1",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Get email of callee": {
      "main": [
        [
          {
            "node": "Prepare payload18",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
```

```
"Create Phone Ownership of Caller4": {
  "main": [
    [
      {
        "node": "Merge1",
        "type": "main",
        "index": 2
      }
    ],
    [
      {
        "node": "Combine all errors1",
        "type": "main",
        "index": 1
      }
    ]
  ]
},
"Search through Agent-MCPClient-LinkedIn-Automation": {
  "main": [
    [
      {
        "node": "JSON Parser",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
        "index": 2
      }
    ]
  ]
},
"OpenAI Chat Model1": {
  "ai_languageModel": [
    [
      {
        "node": "JSON Parser",
        "type": "ai_languageModel",
        "index": 0
      }
    ]
  ],
  "main": [
    []
  ]
},
"Create a contact": {
  "main": [
    [
      {
        "node": "contact create?",
        "type": "main",
        "index": 0
      }
    ],
```

```
      [
        {
          "node": "Combine all errors2",
          "type": "main",
          "index": 4
        }
      ]
    ]
  },
  "contact create?": {
    "main": [
      [
        {
          "node": "Merge",
          "type": "main",
          "index": 3
        }
      ],
      [
        {
          "node": "contact not created in mautic",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "contact not created in mautic": {
    "main": [
      [
        {
          "node": "Merge",
          "type": "main",
          "index": 4
        }
      ]
    ]
  },
  "Search through Agent-MCPClient-LinkedIn-Automation1": {
    "main": [
      [
        {
          "node": "JSON PARSER",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors1",
          "type": "main",
          "index": 2
        }
      ]
    ]
  },
  "OpenAI Chat Model2": {
    "ai_languageModel": [
      [
```

```
        {
          "node": "JSON PARSER",
          "type": "ai_languageModel",
          "index": 0
        }
      ]
    ],
    "main": [
      [
        {
          "node": "Combine all errors1",
          "type": "main",
          "index": 5
        }
      ]
    ]
  },
  "Create a contact1": {
    "main": [
      [
        {
          "node": "contact create?1",
          "type": "main",
          "index": 0
        }
      ],
      [
        {
          "node": "Combine all errors1",
          "type": "main",
          "index": 6
        }
      ]
    ]
  },
  "contact create?1": {
    "main": [
      [
        {
          "node": "Merge1",
          "type": "main",
          "index": 3
        }
      ],
      [
        {
          "node": "contact not created in mautic1",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "contact not created in mautic1": {
    "main": [
      [
        {
          "node": "Merge1",
          "type": "main",
```

```
        "index": 4
      }
    ]
  ]
},
"Get output": {
  "main": [
    [
      {
        "node": "Create a contact",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Structured Output Parser": {
  "ai_outputParser": [
    [
      {
        "node": "JSON Parser",
        "type": "ai_outputParser",
        "index": 0
      }
    ]
  ]
},
"Structured Output Parser1": {
  "ai_outputParser": [
    [
      {
        "node": "JSON PARSER",
        "type": "ai_outputParser",
        "index": 0
      }
    ]
  ]
},
"Set values": {
  "main": [
    [
      {
        "node": "Prepare payload20",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"get output1": {
  "main": [
    [
      {
        "node": "Create a contact1",
        "type": "main",
        "index": 0
      }
    ]
  ]
```

```
},
"COMMUNICATES_WITH Metrics Update1": {
  "main": [
    [
      {
        "node": "Qualify for RAG?",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors1",
        "type": "main",
        "index": 8
      }
    ]
  ]
},
"JSON Parser": {
  "main": [
    [
      {
        "node": "Get output",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors2",
        "type": "main",
        "index": 3
      }
    ]
  ]
},
"JSON PARSER": {
  "main": [
    [
      {
        "node": "get output1",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Combine all errors1",
        "type": "main",
        "index": 4
      }
    ]
  ]
},
"Prepare payload for TOOL-LogEvent": {
  "main": [
    [
      {
```

```
        "node": "Invoke TOOL-LogEvent",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload for TOOL-LogEvent1": {
  "main": [
    [
      {
        "node": "Invoke TOOL-LogEvent1",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Update RAG References on Call node": {
  "main": [
    [
      {
        "node": "Prepare payload for TOOL-LogEvent1",
        "type": "main",
        "index": 0
      }
    ],
    [
      {
        "node": "Prepare payload for TOOL-LogEvent2",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload for TOOL-LogEvent2": {
  "main": [
    [
      {
        "node": "Invoke TOOL-LogEvent1",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Prepare payload for TOOL-LogEvent4": {
  "main": [
    [
      {
        "node": "Invoke TOOL-LogEvent2",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Combine all errors": {
```

```
    "main": [
      [
        {
          "node": "Prepare payload for TOOL-LogEvent4",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Prepare payload for TOOL-LogEvent5": {
    "main": [
      [
        {
          "node": "Invoke TOOL-LogEvent3",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Combine all errors1": {
    "main": [
      [
        {
          "node": "Prepare payload for TOOL-LogEvent5",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Prepare payload for TOOL-LogEvent6": {
    "main": [
      [
        {
          "node": "Invoke TOOL-LogEvent4",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Combine all errors2": {
    "main": [
      [
        {
          "node": "Prepare payload for TOOL-LogEvent6",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Prepare payload for TOOL-LogEvent7": {
    "main": [
      [
        {
          "node": "Invoke TOOL-LogEvent5",
```

```
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Combine all errors3": {
      "main": [
        [
          {
            "node": "Prepare payload for TOOL-LogEvent7",
            "type": "main",
            "index": 0
          }
        ]
      ]
    }
  },
  "active": true,
  "settings": {
    "executionOrder": "v1",
    "binaryMode": "separate",
    "availableInMCP": false
  },
  "versionId": "37121ec5-824a-403a-b1f7-fe165bcf84ea",
  "meta": {
    "templateCredsSetupCompleted": true,
    "instanceId":
"7753fe38446d6250c90cc871e4cbf3f516fb91efe3bd7296ec632cd52f91e6e3"
  },
  "id": "1x62QQE3TZComsyW",
  "tags": [
    {
      "updatedAt": "2025-10-30T06:01:02.852Z",
      "createdAt": "2025-10-30T06:01:02.852Z",
      "id": "qtYu6MRMwdO0zcY9",
      "name": "Webhook"
    }
  ]
}
```